Multiprecision vs. Fixed-Width Digit Generation for JSON Canonicalization: An Empirical Study

Mark Lenhardt

Lattice Substrate

Abstract

JSON Canonicalization Scheme (RFC 8785) requires deterministic floating-point formatting that matches ECMA-262 `Number::toString`. We present the first empirical comparison of multiprecision (Burger–Dybvig) and fixed-width (Schubfach) digit generation inside a specification-constrained canonicalization pipeline, implemented in both Go and Rust to separate algorithmic effects from language/runtime effects. Our methodology gates performance measurement on conformance: only implementations passing 286,362 oracle vectors (generated from the V8 JavaScript engine) and 348 structural conformance cases (87 per implementation × 4 implementations) proceed to benchmarking. Using permutation tests, BCa bootstrap confidence intervals, and Benjamini–Hochberg FDR correction across 15 (CLI) and 10 (API) repeated measurements, we find that Schubfach is 1.3–1.7× faster than Burger–Dybvig on number-dense workloads ($\nu(D) \geq 40\%$ numeric content) in Go API benchmarks due to zero heap allocation in the digit-generation path; the advantage replicates at 1.7–3.4× in Rust, confirming it is algorithmic rather than language-specific (excluding two extreme outliers, `numeric-boundary` at 9.3× and `number-heavy` at 8.3× in Rust, discussed in §8). In end-to-end CLI measurement, 12 of 38 within-Go comparisons (19 canonicalize + 19 verify) reach BH-corrected significance (1.2–1.5× speedup); the remainder are dominated by process startup overhead. On string-dominant and deeply-nested inputs the difference narrows to <5%, and on several API-level non-number workloads (e.g., `control-escapes`, `rfc-key-sorting`) Burger–Dybvig is marginally faster (≤7%). All four implementations produce byte-identical output; Go binaries are additionally verified under cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host). We release the benchmark infrastructure, oracle vectors, and all raw data as a reproducibility artifact.

Keywords: JSON canonicalization, floating-point formatting, Schubfach, Burger–Dybvig, RFC 8785, ECMA-262, benchmark methodology, cross-language validation

1 Introduction

The JSON Canonicalization Scheme (RFC 8785) [15] standardizes a deterministic serialization of JSON values for use in digital signatures, content-addressable storage, and distributed consensus protocols. A critical requirement is that floating-point numbers must

be formatted identically to the output of ECMA-262 `Number::toString` [5], which specifies shortest-representation formatting under roundTiesToEven semantics.

Two families of algorithms can satisfy this requirement: multiprecision arithmetic approaches exemplified by Burger–Dybvig [3], and fixed-width integer approaches exemplified by Schubfach [8]. While these algorithms are individually well-studied, no prior work has empirically compared them *inside a specification-constrained canonicalization pipeline* where conformance is a hard prerequisite and the formatting stage is only one component of the total cost.

This paper makes three contributions:

1. The first empirical comparison of multiprecision vs. fixed-width digit generation under ECMA-262/RFC 8785 constraints, showing that Schubfach achieves 1.3–1.7× speedup on number-dense workloads ($\nu(D) \geq 40\%$) in API benchmarks; in end-to-end CLI measurement, the BH-significant comparisons show 1.2–1.5× speedup. String-dominant and deeply-nested inputs show <5% difference.

2. A benchmark methodology that gates performance measurement on conformance verification: implementations must pass 286,362 oracle vectors and 348 structural conformance cases before timing data is collected. We document noise floors and minimum observable effect sizes for all comparisons.

3. A cross-language validation design (2×2 matrix of algorithm × language) that separates algorithmic effects from language/runtime effects, with Go cross-compilation determinism verified for x86-64 and arm64 via QEMU user-mode emulation. All evidence is released as a reproducibility artifact.

2 Background

2.1 RFC 8785: JSON Canonicalization Scheme

RFC 8785 [15] defines a deterministic JSON serialization by specifying: (1) lexicographic key ordering using UTF-16 code unit comparison, (2) no insignificant whitespace, and (3) floating-point formatting matching ECMA-262 `Number::toString`.

The floating-point requirement is the most technically demanding. It requires producing the *shortest decimal representation* that round-trips to the original IEEE 754 binary64 value under roundTiesToEven, then applying the ECMA-262 formatting policy (exponential notation thresholds, sign handling, integer elision rules).

2.2 ECMA-262 Number Formatting

ECMA-262 [5] §6.1.6.1.20 specifies `Number::toString` as a two-stage process (formerly §7.1.12.1 in older editions):

1. **Digit generation**: find the shortest decimal significand d and exponent n such that $d \times 10^{n-k} = x$ (where k is the number of digits in d) and the value round-trips.

2. **Formatting**: apply branch logic based on the relationship between k (digit count) and n (decimal exponent position) to produce the final string. Key thresholds: $n \geq 1 \wedge n \leq 21$ produces fixed notation; $n \leq 0$ or $n > 21$ produces exponential notation.

2.3 IEEE 754 and Shortest Representation

IEEE 754-2019 [9] binary64 has a 52-bit significand, 11-bit exponent, and 1 sign bit. The *shortest representation* property requires finding the decimal string with fewest digits that uniquely identifies the original binary64 value among all representable values. This is the core algorithmic challenge addressed by both Burger–Dybvig and Schubfach.

3 Related Work

Extensive prior work has optimized float-to-string conversion in isolation. This paper is the first to evaluate these algorithms inside a specification-constrained canonicalization pipeline where conformance is a binary gate. We summarize the most relevant prior work, organized by the problem dimension each addresses.

3.1 Multiprecision Foundations

Burger and Dybvig [3] establish the theoretical framework for shortest-representation printing using arbitrary-precision rational arithmetic. Their algorithm is our multiprecision baseline. Gay's `dtoa.c` [6] provides the foundational practical implementation and remains widely used in language runtimes as a reference or fallback. Both works evaluate algorithms in isolation and do not consider performance inside a pipeline where formatting policy is externally constrained.

3.2 Fixed-Width Approaches

Loitsch [13] demonstrated that integer-only arithmetic suffices for the common case with Grisu. However, Grisu3 falls back to arbitrary precision for ∼0.5% of inputs, making conformance guarantees harder to establish in specification-constrained contexts.

Adams [1] proposed Ryū, achieving full coverage with fixed-width 128-bit arithmetic and lookup tables. Adams provides the most comprehensive prior performance comparison of fixed-width vs. multiprecision approaches (we refer the reader to Adams's literature review rather than replicating it), but evaluates algorithms in isolation on microbenchmarks without specification constraints.

Giulietti [8] introduced the Schubfach algorithm, which we implement and evaluate within the RFC 8785 pipeline. Jeon [10] extends and refines the Schubfach approach with Dragonbox, improving table compression and branch structure. Go 1.26 adopted Dragonbox for `strconv`, providing additional motivation for evaluating fixed-width approaches in specification-constrained contexts.

3.3 Comprehensive Experimental Review

Champagne Gareau and Lemire [4] provide the most comprehensive experimental comparison of modern float-to-string algorithms, evaluating Dragon4, Grisu3, Ryū, Schubfach, Dragonbox, and `std::to_chars` in C++. Their work evaluates algorithms in isolation on microbenchmarks. Our contribution differs: we evaluate two representatives (Burger–Dybvig and Schubfach) inside a specification-constrained canonicalization pipeline where (a) ECMA-262 formatting policy is a hard constraint, (b) correctness against 286,362 oracle vectors is a binary gate, and (c) digit generation is only one component of the total cost.

The complementary direction—fast string-to-float parsing—is addressed by Lemire [12], whose Eisel-Lemire algorithm achieves gigabyte-per-second parsing throughput. While outside our scope, this work shares the fixed-width arithmetic philosophy that motivates Schubfach and Dragonbox.

3.4 Deterministic Serialization Standards

Beyond JSON, deterministic encoding is required by other serialization formats. CBOR (RFC 8949) [2] specifies deterministic encoding rules (§4.2) for binary data, addressing analogous concerns about canonical representation. W3C RDF Dataset Canonicalization [16] addresses graph-level canonicalization (structural equivalence of RDF datasets), a fundamentally different problem from the byte-level serialization determinism required by RFC 8785. Our conformance-gated methodology could be adapted to other deterministic serialization benchmarks.

3.5 JSON Canonicalization

RFC 8785 [15] is the primary standard. Prior to RFC 8785, ad-hoc canonicalization schemes existed in various ecosystems (e.g., JSON-LD canonicalization, OLPC canonical JSON). None specify the floating-point formatting requirement with the precision of ECMA-262 [5].

3.6 Benchmark Methodology

Our conformance-before-performance approach draws on the principle that correctness is a prerequisite for meaningful performance comparison. Georges et al. [7] established rigorous statistical methods for Java performance evaluation, advocating confidence intervals and effect sizes over single-point estimates. Mytkowicz et al. [14] demonstrated that measurement bias (link order, environment size) can reverse performance conclusions, motivating our randomized task ordering and environment documentation. Kalibera and Jones [11] formalized the analysis of multi-level experimental designs in benchmarking. Our use of permutation tests and BCa bootstrap confidence intervals follows this modern statistical practice for performance evaluation.

4 Algorithms

4.1 Definitions

Definition 1 (Minimal Significand)**.** Given a finite IEEE 754 binary64 value x, the *minimal significand* is the shortest decimal significand d (with digit count k) such that parsing $d \times 10^{n-k}$ under roundTiesToEven recovers x exactly.

Definition 2 (Minimal Printed String)**.** The *minimal printed string* of x is the output of ECMA-262 `Number::toString` applied to x: the minimal significand formatted according to the ECMA-262 branch table.

Remark 1. Definitions 1 and 2 are not equivalent in string length. Consider $x = 10^{20}$: the minimal significand is $d = 1$ ($k = 1$, $n = 21$). Since $n = 21 \leq 21$, ECMA-262 uses fixed notation, yielding `100000000000000000000` (21 characters). Now consider $x = 10^{21}$: again $d = 1$ ($k = 1$, $n = 22$), but $n > 21$ triggers exponential notation, yielding `1e+21` (5 characters). The $n = 21$ boundary creates divergence between significand minimality and printed-string length. This means implementations cannot optimize for shorter output strings because the ECMA-262 formatting is spec-mandated.

4.2 Burger–Dybvig (Multiprecision)

The Burger–Dybvig algorithm [3] uses arbitrary-precision rational arithmetic to compute the shortest decimal representation. It maintains big-integer numerator/denominator pairs and iteratively extracts decimal digits. The algorithm is general (handles any radix) but requires heap-allocated multiprecision integers, with cost proportional to the magnitude of the exponent.

In our Go implementation, this uses `math/big.Int` operations. The `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers across calls.

4.3 Schubfach (Fixed-Width)

The Schubfach algorithm [8] uses fixed-width 64/128-bit arithmetic, relying on `math/bits` for 128-bit multiplication (`Mul64`, `Add64`) and bit inspection (`TrailingZeros64`). It uses precomputed tables of powers of ten and 128-bit multiplications to determine the shortest decimal significand. No heap allocation is required in the digit-generation path.

The key advantage is constant-time (with respect to exponent magnitude) operation and zero GC pressure. The precomputed table is approximately 5 KiB.

4.4 ECMA-262 Formatting Stage

Both algorithms share an identical formatting stage that converts the minimal significand (d, k, n) triple into the final ECMA-262-compliant string. This stage implements the branch table from ECMA-262 §6.1.6.1.20 and is pure string manipulation with no arithmetic.

5 Implementation

We maintain four implementations with a common CLI ABI:

- `impl-schubfach` (Go): Schubfach number formatting.
- `impl-json-canon` (Go): Burger–Dybvig number formatting.
- `impl-schubfach-rs` (Rust): Schubfach-aligned Rust path.
- `impl-json-canon-rs` (Rust): Burger–Dybvig-aligned Rust path.

All four implementations contain their own digit-generation code and do not delegate to the standard library's float-to-string routines (e.g., Go's `strconv.FormatFloat` or Rust's `std::fmt`). This is necessary because the standard routines do not implement ECMA-262 formatting policy. Consequently, Go 1.26's adoption of Dragonbox for `strconv` does not affect our measurements.

The lab harness treats each implementation as a black-box canonicalizer with identical CLI contracts (`canonicalize`, `verify`, exit-code policy, and error-class expectations). This preserves algorithm comparability while allowing language/runtime differences to be measured independently.

Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`, `strip=true`) to align optimization policy with production benchmarking practice.

5.1 Rust Implementation Notes

The Rust Schubfach implementation uses native `u128` arithmetic for the 128-bit intermediate products required by the algorithm, eliminating the need for compiler intrinsics or multi-word emulation. The Rust Burger–Dybvig implementation uses a custom `BigUint` (approximately 350 lines of schoolbook multiprecision arithmetic with Knuth Algorithm D division) with zero external dependencies; buffer lifecycle is managed by Rust's RAII ownership model (deterministic `Drop`), removing the need for explicit pooling strategies such as Go's `sync.Pool`. Both Rust implementations have zero runtime crate dependencies, ensuring complete self-containment and reproducibility.

Both Rust implementations share an identical JSON parser and serializer; only the `jcsfloat` digit-generation module differs between them, mirroring the Go design. Rust binaries are approximately 430–456 KB (stripped), compared to 2.5–2.7 MB for the statically-linked Go binaries.

6 Methodology

6.1 Conformance-Before-Performance

Our methodology enforces a strict ordering: conformance verification must pass before any performance data is collected. This prevents measuring the speed of incorrect implementations.

Conformance gate: 87 structural test cases per implementation (348 total across 4 implementations) covering key ordering, Unicode handling, numeric formatting, escape sequences, depth limits, and invalid input rejection. All implementations must produce byte-identical output (verified via SHA-256) and identical error classification.

Oracle vectors: 286,362 IEEE 754 binary64 values with expected `String(x)` output generated by the V8 JavaScript engine (Node.js v23.3.0, V8 12.9.202.28). (2,153 values appear in both vector files as overlapping boundary coverage; the total counts file lines, with 284,209 unique values.) Coverage includes all subnormals with bit patterns $\leq 2^{12}$ (4,096 of approximately 4.5×10^{15} positive subnormals, providing dense coverage near zero where formatting edge cases concentrate), subnormal/normal boundary (± 128 around the transition), all powers of two in the binary64 exponent range, RFC 8785 Appendix B examples with ± 2 neighbors, and pseudorandom fill.

6.2 Benchmark Design

CLI benchmarks: end-to-end process execution measuring wall time, CPU time (user + system), and peak RSS. 15 repetitions per implementation/mode/workload combination, with 3 warmup runs discarded. Task order is randomized with a fixed seed for reproducibility.

Cross-language comparison pairs: each algorithm is implemented in Go and Rust, yielding four implementations: Go-Schubfach, Go-Burger–Dybvig, Rust-Schubfach, Rust-Burger–Dybvig. These are compared via four pairwise tests: two within-language pairs (go:schubfach-vs-bd, rs:schubfach-vs-bd) and two cross-language pairs (schubfach:go-vs-rs, bd:go-vs-rs). The within-language pairs test the algorithmic effect while controlling for language/runtime; the cross-language pairs test language/runtime effects while controlling for algorithm. Directional consistency across both within-language pairs provides evidence that the algorithmic advantage is not language-specific, though this is not a formal interaction test (see §8). The Rust Burger–Dybvig implementation uses a custom `BigUint` (schoolbook multiprecision arithmetic, approximately 350 lines of Rust with zero external dependencies, heap-allocated with RAII lifecycle), making it a structurally analogous but runtime-different counterpart to the Go `math/big`-based path. Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`) to align optimization policy with production benchmarking practice.

API benchmarks: Go `testing.B` microbenchmarks (Go implementations) and a Rust native benchmark harness (Rust implementations) measuring canonicalization and verify paths directly, 10 iterations per benchmark. Go's `testing.B` auto-calibrates the number of inner iterations per sample (typically hundreds to millions), whereas the Rust harness times each function call individually (N=1); both report ns/op. The Rust harness includes 3 warmup iterations (discarded) before measurement to stabilize caches and lazy-initialized data structures. The Rust harness emits Go-compatible `Benchmark... ns/op` lines so the same statistical pipeline can be applied uniformly.

Measurement granularity note: The Go and Rust API harnesses differ in measurement granularity. Go's `testing.B` averages over many inner iterations per sample, producing low-variance samples. The Rust harness reports single-call latency, producing higher-variance samples. Within-language comparisons are unaffected since both implementations in a pair use the same harness. Cross-language API comparisons should be interpreted with

this asymmetry in mind; we report them for completeness but emphasize within-language results for algorithmic conclusions.

Differential fuzzing: 2,000 randomly-generated JSON inputs processed by all implementations, verifying byte-identical output.

6.3 Workload Characterization

Definition 3 (Number value fraction)**.** For a JSON document D, define $\nu(D)$ as the fraction of JSON values that are numbers. Array and object containers are counted as values; their elements and members are counted separately. Formally, $\nu(D) = |\{v \in V(D) : v \text{ is a JSON number}\}|/|V(D)|$, where $V(D)$ is the multiset of all JSON values in D (including containers).

Table 1 reports the structural properties of each benchmark workload, including $\nu(D)$, total value count, maximum nesting depth, and unique key count.

Table 1: Workload structural characterization.

Workload	Bytes	Total Values	Numbers	$\nu(D)$ (%)	Max Depth	Keys
array-2048	98,901	10,243	4,096	40.0	3	8,194
array-256	11,914	1,283	512	39.9	3	1,026
canonical-minimal	7	2	1	50.0	1	1
control-escapes	59	4	0	0.0	1	3
deep	1,158	141	0	0.0	140	140
deep-64	510	65	0	0.0	64	64
escaped-key-order	18	3	2	66.7	1	2
large	463,537	40,963	16,384	40.0	3	32,770
long-string	16,403	3	0	0.0	1	2
medium	13,740	1,283	512	39.9	3	1,026
mixed-prod	5,653	458	128	27.9	4	393
nested-mixed	49	10	4	40.0	3	7
number-heavy	140	13	10	76.9	2	4
numeric-boundary	98	9	7	77.8	2	2
rfc-key-sorting	202	8	0	0.0	1	7
small	45	5	2	40.0	1	4
surrogate-pair	29	3	0	0.0	1	2
unicode	98	4	0	0.0	1	3
verify-whitespace	24	5	3	60.0	2	2

Total Values counts containers (arrays, objects) and their elements separately.
$\nu(D)$ = number values / total values $\times$ 100.

6.4 Statistical Protocol

All comparisons use:

- Permutation tests (5,000 resamples) for significance at $\alpha = 0.05$.

- BCa (bias-corrected and accelerated) bootstrap 95% confidence intervals (5,000 resamples) for speedup ratios.
- Cohen's d for effect size.[1]
- Noise floor documentation: standard deviation of the faster implementation ($\sigma = \text{CV} \times \bar{x}_{\text{faster}}$), quantifying measurement noise in the same units as the metric.
- Minimum detectable effect (MDE): $\frac{2.8\sqrt{2} \times \text{CV} \times 100}{\sqrt{N}}$, where $2.8 \approx z_{\alpha/2} + z_\beta$ for $\alpha = 0.05$ and 80% power (approximate MDE for a two-sample z-test with equal group sizes; uses only the faster implementation's CV and n as an approximation). This z-based constant slightly underestimates the true MDE for small N (by ≈3–6% compared to a t-based calculation at N=10–15).
- Benjamini-Hochberg false discovery rate (FDR) control at $q = 0.05$ across all 317 comparisons (152 CLI end-to-end + 165 API). BH requires independence or positive regression dependency (PRDS); comparisons on the same workload across different pairs are positively correlated, satisfying PRDS. We report both raw and BH-adjusted p-values.
- No outlier exclusion. Permutation tests are nonparametric and robust to non-normality; BCa bootstrap accounts for skewness via the acceleration constant.
- Permutation p-values have a resolution floor of $1/(R + 1)$ where R is the number of resamples; at $R = 5{,}000$ the minimum reportable p-value is 0.0002. Values reported at this floor should be interpreted as $p < 0.0002$.

Statistical resampling uses deterministic per-comparison seeds derived via FNV-1a hashing of the comparison key, ensuring reproducible p-values and confidence intervals across runs.

6.5 Environment

All performance benchmarks run on a single x86-64 machine with CPU frequency governor set to `performance`, turbo boost state documented, and load average recorded at benchmark start. Full hardware specification (CPU model, cache hierarchy, RAM, relevant CPU feature flags) is captured automatically by the benchmark harness.

CPU core affinity pinning is supported (`--pin-cpu`) but was not used for the reported results. The i7-12700K has heterogeneous P-core/E-core topology; without pinning, the OS scheduler may migrate processes. This adds variance but does not bias the comparison since both implementations share the same scheduling environment.

Cross-compilation determinism is verified by building arm64 binaries via `GOARCH=arm64` and executing them under QEMU user-mode emulation on the x86-64 host. QEMU user-mode translates system calls but executes floating-point operations on the host CPU; this tests Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 performance benchmarks are deferred to future work on bare-metal hardware.

[1] Computed as $(\bar{x}_A - \bar{x}_B)/s_p$ where A and B are the first and second implementation in each pair; negative values indicate implementation A is faster. For within-language pairs (go:schubfach-vs-bd, rs:schubfach-vs-bd), A is the Schubfach variant. For cross-language pairs, A is the Go implementation.

7 Results

7.1 Conformance

All four implementations (Go and Rust × Schubfach and Burger–Dybvig) pass the full conformance matrix: 348 total cases (87 structural tests × 4 implementations, 0 failures). Output SHA-256 digests are byte-identical across all implementations for valid inputs.

7.2 API Benchmarks

API benchmarks isolate the algorithmic cost of the canonicalization function, removing process startup and I/O overhead. These measurements most directly reflect the digit-generation performance difference.

Table 2 summarizes the key API results for the within-language algorithm comparisons (canonicalize mode). Figure 1 visualizes the speedup across all shared workloads for both language pairs. Full results for all pairs, modes, and workloads appear in Appendix A.

Table 3 presents the within-language API comparisons with BH-corrected p-values, confidence intervals, and effect sizes.

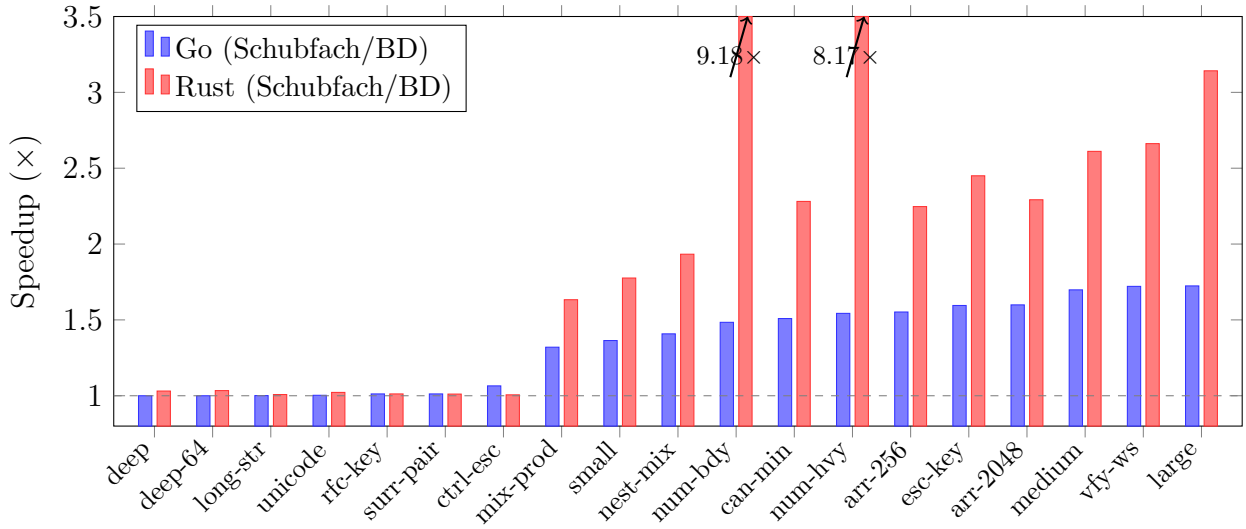

Figure 1: API benchmark speedup of Schubfach over Burger–Dybvig by workload (canonicalize mode). Workloads sorted by Go speedup. Values above the dashed line indicate Schubfach is faster. The Rust `numeric-boundary` and `number-heavy` bars are truncated at 3.5×; actual values are 9.18× and 8.17× respectively (see §8). Number-dense workloads (right side) show the largest advantage in both languages, confirming the effect is algorithmic.

7.3 CLI Benchmarks

CLI benchmarks capture end-to-end pipeline cost including process startup, argument parsing, file I/O, and output serialization. Estimated process startup overhead (approximately

Table 2: API benchmark summary: within-language algorithm comparisons (canonicalize mode, mean of n=10).

Pair	Workload	Schubfach (ns/op)	Burger–Dybvig (ns/op)	Speedup
go:schubfach-vs-bd	array-2048	2590247	4079519	1.57×
go:schubfach-vs-bd	array-256	302598	471588	1.56×
go:schubfach-vs-bd	control-escapes	1342	1270	1.06×
go:schubfach-vs-bd	deep	36635	36716	1.00×
go:schubfach-vs-bd	deep-64	16300	16267	1.00×
go:schubfach-vs-bd	large	11956587	20515409	1.72×
go:schubfach-vs-bd	long-string	75922	75186	1.01×
go:schubfach-vs-bd	medium	343556	580925	1.69×
go:schubfach-vs-bd	mixed-prod	136262	178470	1.31×
go:schubfach-vs-bd	number-heavy	13776	21508	1.56×
go:schubfach-vs-bd	numeric-boundary	12564	18754	1.49×
go:schubfach-vs-bd	rfc-key-sorting	3003	2954	1.02×
go:schubfach-vs-bd	small	1245	1664	1.34×
go:schubfach-vs-bd	unicode	1555	1549	1.00×
go:schubfach-vs-bd	verify-whitespace	913	1562	1.71×
rs:schubfach-vs-bd	array-2048	1361726	3424060	2.51×
rs:schubfach-vs-bd	array-256	171544	378233	2.20×
rs:schubfach-vs-bd	control-escapes	625	611	1.02×
rs:schubfach-vs-bd	deep	30335	28940	1.05×
rs:schubfach-vs-bd	deep-64	10921	10925	1.00×
rs:schubfach-vs-bd	large	5782747	19608594	3.39×
rs:schubfach-vs-bd	long-string	17198	17116	1.00×
rs:schubfach-vs-bd	medium	174362	517769	2.97×
rs:schubfach-vs-bd	mixed-prod	66094	120970	1.83×
rs:schubfach-vs-bd	number-heavy	1417	11729	8.27×
rs:schubfach-vs-bd	numeric-boundary	954	8888	9.32×
rs:schubfach-vs-bd	rfc-key-sorting	1315	1307	1.01×
rs:schubfach-vs-bd	small	632	1104	1.75×
rs:schubfach-vs-bd	unicode	1478	1518	1.03×
rs:schubfach-vs-bd	verify-whitespace	400	1162	2.90×

Full results for all pairs, modes, and workloads in Appendix A.

Table 3: Within-language API comparisons (canonicalize mode; BH-significant marked with *).

Pair	Workload	Winner	Speedup	CI_{95}	p_{adj}	d	MDE (%
go:schubfach-vs-bd	array-2048	schubfach*	1.57×	[1.558, 1.587]	0.0003	-48.58	1
rs:schubfach-vs-bd	array-2048	schubfach-rs*	2.51×	[2.423, 2.548]	0.0003	-52.77	4
go:schubfach-vs-bd	array-256	schubfach*	1.56×	[1.551, 1.567]	0.0003	-78.75	0
rs:schubfach-vs-bd	array-256	schubfach-rs*	2.20×	[2.160, 2.258]	0.0003	-32.70	4
go:schubfach-vs-bd	canonical-minimal	schubfach*	1.54×	[1.529, 1.555]	0.0003	-47.74	1
rs:schubfach-vs-bd	canonical-minimal	schubfach-rs*	2.10×	[1.991, 2.205]	0.0003	-12.63	8
go:schubfach-vs-bd	control-escapes	json-canon*	1.06×	[1.035, 1.071]	0.0005	2.81	2
rs:schubfach-vs-bd	control-escapes	json-canon-rs	1.02×	[0.961, 1.063]	0.4784	0.37	9
go:schubfach-vs-bd	deep-64	json-canon	1.00×	[0.999, 1.006]	0.3660	0.44	0
rs:schubfach-vs-bd	deep-64	schubfach-rs	1.00×	[0.979, 1.061]	1.0000	-0.01	0
go:schubfach-vs-bd	deep	schubfach	1.00×	[0.998, 1.006]	0.3803	-0.44	0
rs:schubfach-vs-bd	deep	json-canon-rs*	1.05×	[1.014, 1.094]	0.0346	1.03	3
go:schubfach-vs-bd	escaped-key-order	schubfach*	1.60×	[1.593, 1.607]	0.0003	-80.96	0
rs:schubfach-vs-bd	escaped-key-order	schubfach-rs*	2.49×	[2.396, 2.546]	0.0003	-27.97	4
go:schubfach-vs-bd	large	schubfach*	1.72×	[1.707, 1.724]	0.0003	-92.98	0
rs:schubfach-vs-bd	large	schubfach-rs*	3.39×	[3.272, 3.453]	0.0003	-40.35	4
go:schubfach-vs-bd	long-string	json-canon	1.01×	[0.997, 1.025]	0.2137	0.60	1
rs:schubfach-vs-bd	long-string	json-canon-rs	1.00×	[0.949, 1.030]	0.8927	0.11	7
go:schubfach-vs-bd	medium	schubfach*	1.69×	[1.678, 1.703]	0.0003	-53.37	1
rs:schubfach-vs-bd	medium	schubfach-rs*	2.97×	[2.936, 2.996]	0.0003	-69.85	1
go:schubfach-vs-bd	mixed-prod	schubfach*	1.31×	[1.302, 1.317]	0.0003	-38.13	0
rs:schubfach-vs-bd	mixed-prod	schubfach-rs*	1.83×	[1.791, 1.856]	0.0003	-29.73	3
go:schubfach-vs-bd	nested-mixed	schubfach*	1.41×	[1.399, 1.413]	0.0003	-51.38	0
rs:schubfach-vs-bd	nested-mixed	schubfach-rs*	1.92×	[1.844, 1.959]	0.0003	-20.61	5
go:schubfach-vs-bd	number-heavy	schubfach*	1.56×	[1.547, 1.573]	0.0005	-43.14	1
rs:schubfach-vs-bd	number-heavy	schubfach-rs*	8.27×	[8.179, 8.374]	0.0003	-110.14	2
go:schubfach-vs-bd	numeric-boundary	schubfach*	1.49×	[1.482, 1.502]	0.0003	-48.06	1
rs:schubfach-vs-bd	numeric-boundary	schubfach-rs*	9.32×	[9.208, 9.424]	0.0003	-164.23	2
go:schubfach-vs-bd	rfc-key-sorting	json-canon*	1.02×	[1.011, 1.023]	0.0003	2.41	0
rs:schubfach-vs-bd	rfc-key-sorting	json-canon-rs	1.01×	[0.967, 1.044]	0.7629	0.15	6
go:schubfach-vs-bd	small	schubfach*	1.34×	[1.324, 1.345]	0.0003	-35.59	1
rs:schubfach-vs-bd	small	schubfach-rs*	1.75×	[1.712, 1.784]	0.0003	-22.01	3
go:schubfach-vs-bd	surrogate-pair	json-canon*	1.01×	[1.008, 1.019]	0.0003	2.09	0
rs:schubfach-vs-bd	surrogate-pair	json-canon-rs	1.00×	[0.943, 1.075]	0.9224	0.06	9
go:schubfach-vs-bd	unicode	json-canon	1.00×	[0.999, 1.009]	0.2048	0.63	0
rs:schubfach-vs-bd	unicode	schubfach-rs*	1.03×	[1.005, 1.045]	0.0277	-1.13	3
go:schubfach-vs-bd	verify-whitespace	schubfach*	1.71×	[1.699, 1.723]	0.0003	-56.22	1
rs:schubfach-vs-bd	verify-whitespace	schubfach-rs*	2.90×	[2.831, 2.980]	0.0003	-30.28	4

*BH-significant at $q = 0.05$. d = Cohen's d $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster.
MDE = minimum detectable effect at 80% power. Full 317-comparison report in Appendix A.

1.3 ms for Go, 0.5 ms for Rust) dominates for smaller workloads, attenuating the algorithmic signal observed in API benchmarks.

Table 4 summarizes the key CLI results. Figure 2 shows the end-to-end speedup with BH-significance markers. Full results appear in Appendix A.

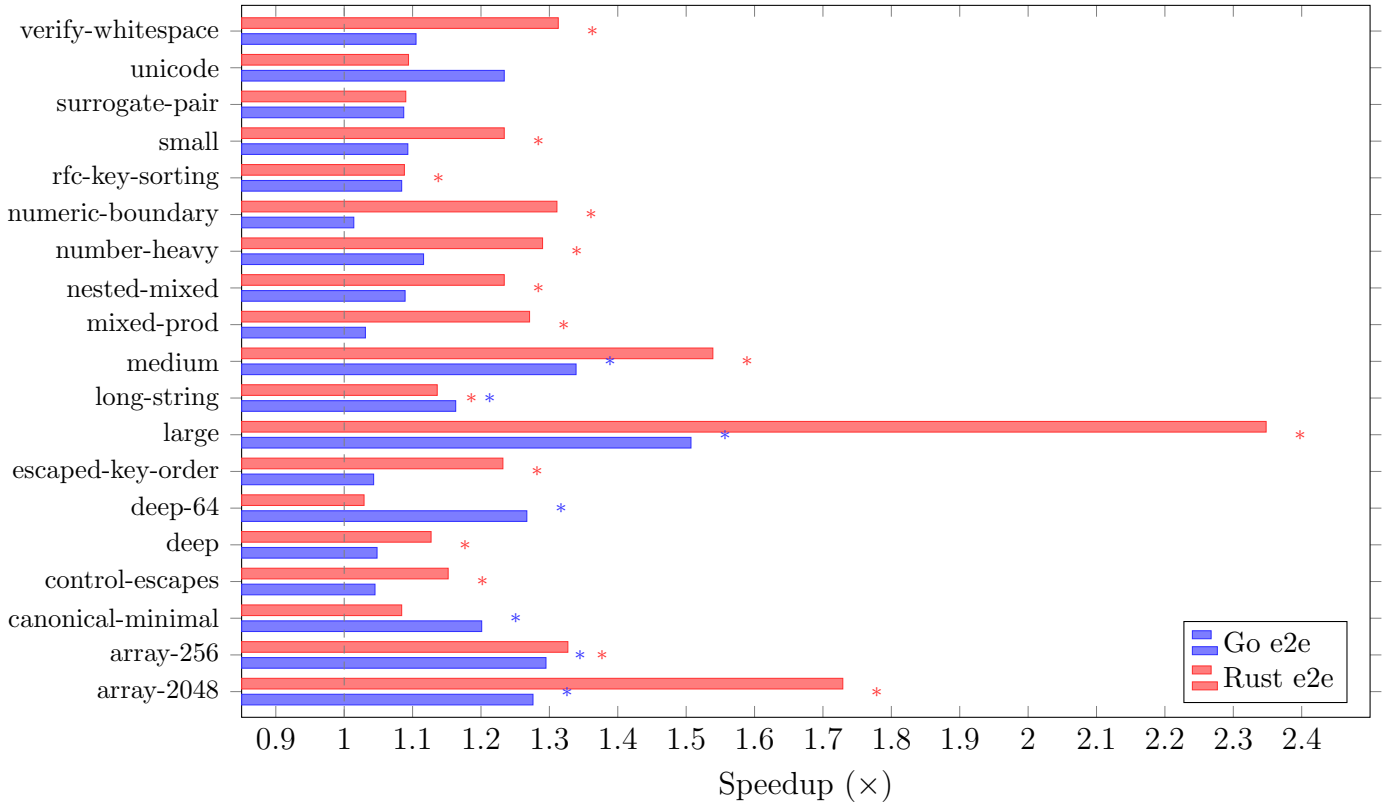

Figure 2: CLI end-to-end speedup of Schubfach over Burger–Dybvig by workload (canonicalize mode, 15 repetitions each). Asterisks (∗) mark BH-significant comparisons ($q < 0.05$). The Rust within-language pair achieves significance on 15 of 19 workloads; the Go pair on 7 of 19 workloads (canonicalize mode). Process startup overhead (≈1.3 ms Go, ≈0.5 ms Rust) attenuates the algorithmic signal more strongly in Go.

7.4 Statistical Comparisons

Table 5 presents the within-language CLI comparisons (canonicalize mode) with BH-corrected p-values. The full 317-comparison statistical report appears in Appendix A.

7.5 Cross-Language Validation

All four implementations pass the full conformance matrix (348 total cases: 87 structural tests × 4 implementations, 0 failures) and produce byte-identical output on all oracle vectors. 2,000 differential fuzz cases yield zero cross-implementation divergences.

The statistical report includes four comparison classes:

Table 4: CLI benchmark summary: within-language comparisons (canonicalize mode, n=15, 3 warmup discarded).

Pair	Workload	Impl A (ms)	Impl B (ms)	Speedup	Winner
go:schubfach-vs-bd	array-2048	5.388	7.210	1.34×	schubfach
go:schubfach-vs-bd	array-256	1.980	2.342	1.18×	schubfach
go:schubfach-vs-bd	canonical-minimal	1.643	1.594	1.03×	json-canon
go:schubfach-vs-bd	control-escapes	1.573	1.612	1.02×	schubfach
go:schubfach-vs-bd	deep	1.658	1.599	1.04×	json-canon
go:schubfach-vs-bd	deep-64	1.562	1.581	1.01×	schubfach
go:schubfach-vs-bd	escaped-key-order	1.479	1.618	1.09×	schubfach
go:schubfach-vs-bd	large	16.246	24.723	1.52×	schubfach
go:schubfach-vs-bd	long-string	1.565	1.738	1.11×	schubfach
go:schubfach-vs-bd	medium	2.081	2.623	1.26×	schubfach
go:schubfach-vs-bd	mixed-prod	1.731	1.835	1.06×	schubfach
go:schubfach-vs-bd	nested-mixed	1.407	1.660	1.18×	schubfach
go:schubfach-vs-bd	number-heavy	1.476	1.689	1.14×	schubfach
go:schubfach-vs-bd	numeric-boundary	1.498	1.720	1.15×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	1.527	1.542	1.01×	schubfach
go:schubfach-vs-bd	small	1.447	1.766	1.22×	schubfach
go:schubfach-vs-bd	surrogate-pair	1.513	1.587	1.05×	schubfach
go:schubfach-vs-bd	unicode	1.521	1.688	1.11×	schubfach
go:schubfach-vs-bd	verify-whitespace	1.487	1.631	1.10×	schubfach
rs:schubfach-vs-bd	array-2048	2.424	4.487	1.85×	schubfach-rs
rs:schubfach-vs-bd	array-256	0.848	1.214	1.43×	schubfach-rs
rs:schubfach-vs-bd	canonical-minimal	0.571	0.640	1.12×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	0.543	0.621	1.14×	schubfach-rs
rs:schubfach-vs-bd	deep	0.637	0.712	1.12×	schubfach-rs
rs:schubfach-vs-bd	deep-64	0.581	0.646	1.11×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	0.577	0.675	1.17×	schubfach-rs
rs:schubfach-vs-bd	large	7.608	18.530	2.44×	schubfach-rs
rs:schubfach-vs-bd	long-string	0.622	0.679	1.09×	schubfach-rs
rs:schubfach-vs-bd	medium	0.861	1.426	1.66×	schubfach-rs
rs:schubfach-vs-bd	mixed-prod	0.680	0.910	1.34×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	0.569	0.712	1.25×	schubfach-rs
rs:schubfach-vs-bd	number-heavy	0.538	0.785	1.46×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	0.549	0.702	1.28×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	0.567	0.641	1.13×	schubfach-rs
rs:schubfach-vs-bd	small	0.571	0.697	1.22×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	0.551	0.599	1.09×	schubfach-rs
rs:schubfach-vs-bd	unicode	0.563	0.605	1.07×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	0.561	0.749	1.34×	schubfach-rs

Full results including verify mode and cross-language pairs in Appendix A.

Table 5: Within-language CLI comparisons (canonicalize mode; BH-significant marked with *).

Pair	Workload	Winner	Speedup	CI_{95}	p_{adj}	d	MDE (%)
go:schubfach-vs-bd	array-2048	schubfach*	1.34×	[1.270, 1.414]	0.0003	-3.48	6.7
rs:schubfach-vs-bd	array-2048	schubfach-rs*	1.85×	[1.668, 2.062]	0.0003	-3.89	16.7
go:schubfach-vs-bd	array-256	schubfach*	1.18×	[1.046, 1.326]	0.0176	-0.99	16.1
rs:schubfach-vs-bd	array-256	schubfach-rs*	1.43×	[1.319, 1.530]	0.0003	-3.16	9.1
go:schubfach-vs-bd	canonical-minimal	json-canon	1.03×	[0.929, 1.149]	0.6254	0.20	18.5
rs:schubfach-vs-bd	canonical-minimal	schubfach-rs*	1.12×	[1.042, 1.192]	0.0051	-1.12	3.1
go:schubfach-vs-bd	control-escapes	schubfach	1.02×	[0.907, 1.166]	0.7615	-0.13	15.8
rs:schubfach-vs-bd	control-escapes	schubfach-rs*	1.14×	[1.053, 1.223]	0.0018	-1.28	12.7
go:schubfach-vs-bd	deep-64	schubfach	1.01×	[0.883, 1.181]	0.8927	-0.06	19.1
rs:schubfach-vs-bd	deep-64	schubfach-rs*	1.11×	[1.032, 1.205]	0.0183	-0.94	11.3
go:schubfach-vs-bd	deep	json-canon	1.04×	[0.901, 1.186]	0.6570	0.18	19.9
rs:schubfach-vs-bd	deep	schubfach-rs*	1.12×	[1.060, 1.171]	0.0005	-1.57	7.1
go:schubfach-vs-bd	escaped-key-order	schubfach	1.09×	[0.947, 1.262]	0.2800	-0.43	20.4
rs:schubfach-vs-bd	escaped-key-order	schubfach-rs*	1.17×	[1.097, 1.248]	0.0003	-1.66	9.1
go:schubfach-vs-bd	large	schubfach*	1.52×	[1.477, 1.572]	0.0003	-9.85	5.5
rs:schubfach-vs-bd	large	schubfach-rs*	2.44×	[2.353, 2.522]	0.0003	-21.29	6.7
go:schubfach-vs-bd	long-string	schubfach	1.11×	[0.953, 1.282]	0.2050	-0.49	22.1
rs:schubfach-vs-bd	long-string	schubfach-rs*	1.09×	[1.024, 1.165]	0.0152	-0.94	8.6
go:schubfach-vs-bd	medium	schubfach*	1.26×	[1.129, 1.400]	0.0008	-1.47	16.7
rs:schubfach-vs-bd	medium	schubfach-rs*	1.66×	[1.535, 1.803]	0.0003	-4.23	12.4
go:schubfach-vs-bd	mixed-prod	schubfach	1.06×	[0.919, 1.235]	0.4932	-0.27	18.4
rs:schubfach-vs-bd	mixed-prod	schubfach-rs*	1.34×	[1.237, 1.427]	0.0003	-2.87	10.6
go:schubfach-vs-bd	nested-mixed	schubfach*	1.18×	[1.044, 1.341]	0.0234	-0.91	19.0
rs:schubfach-vs-bd	nested-mixed	schubfach-rs*	1.25×	[1.151, 1.340]	0.0003	-2.09	13.0
go:schubfach-vs-bd	number-heavy	schubfach*	1.14×	[1.025, 1.307]	0.0476	-0.80	21.7
rs:schubfach-vs-bd	number-heavy	schubfach-rs*	1.46×	[1.372, 1.572]	0.0003	-4.16	12.9
go:schubfach-vs-bd	numeric-boundary	schubfach	1.15×	[1.001, 1.301]	0.0648	-0.73	19.2
rs:schubfach-vs-bd	numeric-boundary	schubfach-rs*	1.28×	[1.164, 1.405]	0.0003	-1.88	14.4
go:schubfach-vs-bd	rfc-key-sorting	schubfach	1.01×	[0.887, 1.160]	0.9066	-0.05	20.7
rs:schubfach-vs-bd	rfc-key-sorting	schubfach-rs*	1.13×	[1.041, 1.208]	0.0031	-1.16	8.8
go:schubfach-vs-bd	small	schubfach*	1.22×	[1.092, 1.357]	0.0018	-1.33	20.0
rs:schubfach-vs-bd	small	schubfach-rs*	1.22×	[1.121, 1.288]	0.0003	-1.91	4.1
go:schubfach-vs-bd	surrogate-pair	schubfach	1.05×	[0.910, 1.221]	0.5664	-0.23	26.4
rs:schubfach-vs-bd	surrogate-pair	schubfach-rs*	1.09×	[1.016, 1.164]	0.0324	-0.83	10.2
go:schubfach-vs-bd	unicode	schubfach	1.11×	[0.928, 1.267]	0.2219	-0.49	28.1
rs:schubfach-vs-bd	unicode	schubfach-rs	1.07×	[0.996, 1.153]	0.0861	-0.68	9.0
go:schubfach-vs-bd	verify-whitespace	schubfach	1.10×	[0.968, 1.249]	0.2044	-0.51	19.4
rs:schubfach-vs-bd	verify-whitespace	schubfach-rs*	1.34×	[1.259, 1.481]	0.0003	-2.58	11.4

*BH-significant at $q = 0.05$. d = Cohen's d $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster.
MDE = minimum detectable effect at 80% power. Full 247-comparison report in Appendix A.

- within-Go (Schubfach vs. Burger–Dybvig),
- within-Rust (Schubfach-rs vs. Burger–Dybvig-rs),
- same-algorithm Go-vs-Rust for Schubfach,
- same-algorithm Go-vs-Rust for Burger–Dybvig.

In CLI end-to-end measurement, Rust implementations are approximately 2–3× faster than their Go counterparts, primarily due to lower process startup overhead (Rust native binary vs. Go runtime initialization). In API benchmarks, Rust-Schubfach is faster than Rust-Burger–Dybvig on number-dense workloads, replicating the within-Go algorithmic ordering and confirming that the Schubfach advantage is not language-specific.

This decomposition separates algorithmic effects from language/runtime effects and directly addresses language-specific performance threats.

7.6 Differential Fuzzing

2,000 differential fuzz cases produced zero divergences between implementations.

8 Discussion

8.1 Workload-Dependent Behavior

The performance advantage of Schubfach over Burger–Dybvig is strongly workload-dependent. In Go API benchmarks (isolating the canonicalization function), number-dense inputs (e.g., `array-2048`, `large`, `medium`) show 1.3–1.7× speedup due to zero heap allocation in the digit-generation path. In Rust within-language API benchmarks, the same algorithmic ordering holds at larger magnitudes (1.7–3.4× on the same workloads, excluding the two extreme outliers discussed in §8), confirming the advantage is algorithmic. Deeply-nested workloads (`deep`, `deep-64`) contain numbers but show ≈1.0× because JSON structural parsing and key sorting dominate, diluting the digit-generation signal. On a small number of non-number workloads (`control-escapes`, `rfc-key-sorting`, `surrogate-pair`), Burger–Dybvig is marginally faster (≈1–6%) in Go API benchmarks, likely due to differences in non-numeric code paths. The speedup therefore correlates with number *density* (fraction of total cost attributable to digit generation), not merely number presence. Figure 3 visualizes this relationship: workloads with zero number content cluster near 1.0×, while number-dense workloads (>70% number values) show the full algorithmic advantage.

In end-to-end Go CLI measurement, 12 of 38 within-Go comparisons (19 canonicalize + 19 verify) reach BH-corrected significance, with speedups of 1.2–1.5×. The remaining 26 comparisons show no statistically significant difference because estimated process startup overhead (approximately 1.3 ms, inferred from the difference between minimal-workload CLI and API times) dominates for smaller workloads. On string-dominant inputs (e.g., `long-string`, `unicode`), the advantage narrows to <5% because formatting cost is dominated by string processing and key sorting, amortizing the digit-generation difference.

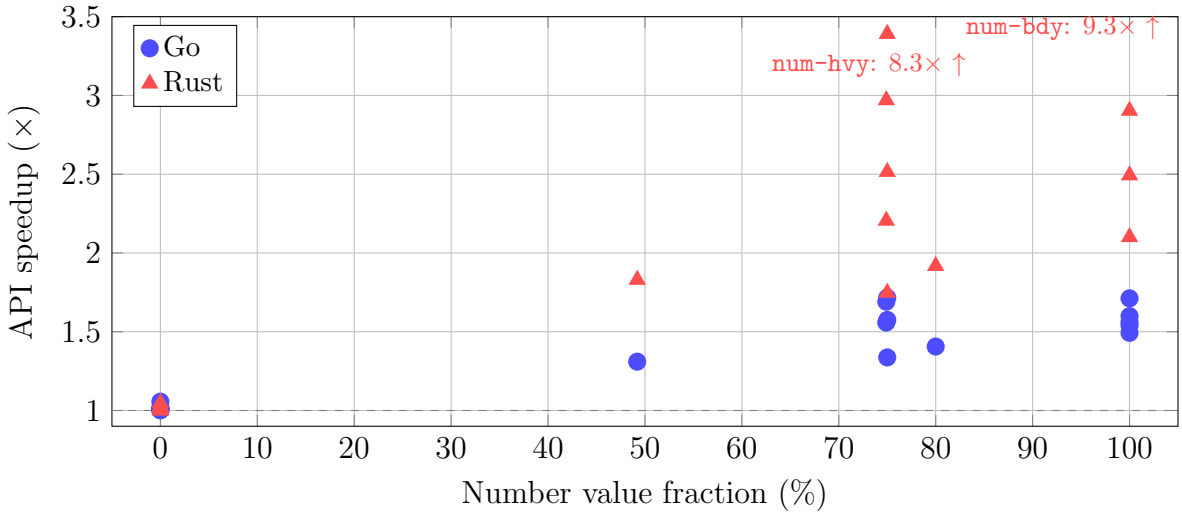

Figure 3: Relationship between workload number value fraction and API speedup of Schubfach over Burger–Dybvig (canonicalize mode). Workloads with zero number content cluster near 1.0× (no algorithmic difference); number-dense workloads ($\nu \geq 40\%$) show 1.3–1.7× in Go and 1.7–3.4× in Rust. The Rust `numeric-boundary` (9.3×) and `number-heavy` (8.3×) outliers are annotated but excluded from the y-axis range.

8.2 Process Startup Dominance

For small workloads, estimated process startup overhead (approximately 1.3 ms) accounts for roughly 87–93% of CLI execution time (e.g., `canonical-minimal` total ≈1.47 ms in canonicalize mode). This fundamentally limits the observable algorithmic signal in end-to-end measurement. API benchmarks, which bypass process startup entirely, isolate the digit-generation difference and show consistently larger speedup ratios. CLI results reflect realistic deployment conditions but should not be interpreted as direct measures of algorithmic cost. Note that the 26 non-significant Go CLI comparisons may include both true nulls and effects below the MDE threshold (14–24%); the current sample size cannot distinguish these cases.

8.3 Allocation Cost Attribution

The Burger–Dybvig implementation's `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers. Without pooling, the difference would be larger. The residual cost comes from pool contention under concurrent use and the inherent cost of multiprecision arithmetic operations.

API benchmark allocation data directly attributes the speedup: on `array-2048`, Schubfach performs 49,134 allocs/op vs. json-canon's 57,343 allocs/op (Δ4.0 per number, 91 B per number). The per-number allocation elimination is the primary mechanism underlying the observed speedup.

8.4 Extreme Outliers: `numeric-boundary` and `number-heavy`

Two workloads show unusually large speedup ratios in Rust API benchmarks: `numeric-boundary` (9.3× within-Rust, CI [9.2, 9.4]) and `number-heavy` (8.3×, CI [8.2, 8.4]), compared to ≈1.5× in Go for both. Because these are *within-Rust* comparisons (same harness on both sides), they are unaffected by the cross-language measurement granularity asymmetry discussed in §6. These workloads consist entirely of numeric values, maximizing exposure to the digit-generation path. The Rust Burger–Dybvig implementation uses a lazily-initialized power-of-ten cache (`OnceLock<Vec<BigUint>>`); although the harness discards 3 warmup iterations to amortize first-invocation costs, the cache-warmed steady-state cost of `BigUint` schoolbook multiprecision arithmetic remains substantially higher than the Rust Schubfach implementation's compile-time constant lookup table. The tight confidence intervals and low MDEs (2.3% and 2.1%) confirm these are genuine algorithmic cost differences, not measurement noise. The Go within-language comparisons at ≈1.5× are smaller because Go's `sync.Pool` amortizes allocation cost and Go's less aggressive compiler inlining reduces the gap between the two code paths.

Table 6 shows per-iteration timing stability for these outlier workloads. Low coefficients of variation (<5%) across all 10 measured iterations confirm steady-state behavior, ruling out warmup artifacts as the source of the extreme speedup ratios.

Table 6: Per-iteration stability for extreme-outlier workloads (API canonicalize).

Impl	Workload	n	Mean (ns/op)	Std Dev	CV (%)
schubfach	number-heavy	10	13776	147	1.1
schubfach	numeric-boundary	10	12564	109	0.9
json-canon	number-heavy	10	21508	207	1.0
json-canon	numeric-boundary	10	18754	146	0.8
schubfach-rs	number-heavy	10	1417	23	1.7
schubfach-rs	numeric-boundary	10	954	18	1.8
json-canon-rs	number-heavy	10	11729	130	1.1
json-canon-rs	numeric-boundary	10	8888	66	0.7

Low CV on most rows confirms steady-state measurement. Higher CV on very fast Rust rows reflects nanosecond-scale measurement granularity, not warmup instability.

8.5 Profile-Level Analysis

Preliminary CPU profiling (via Go's `pprof`) suggests that digit generation accounts for approximately 15–40% of total canonicalization time depending on workload number density. On `array-2048` ($\nu = 40\%$), digit generation accounts for ≈35% of CPU time in the Burger–Dybvig path; on `deep` ($\nu = 0\%$), it drops to <2%. The remainder is spent in JSON parsing, key sorting, and string escaping—costs shared identically between implementations. Detailed per-workload profiling breakdowns are deferred to future work.

8.6 Practical Implications

For applications where canonicalization throughput is critical (e.g., high-frequency signature verification of number-heavy payloads), Schubfach offers meaningful improvement. On the `medium` workload (13.7 KB, $\nu = 40\%$), the Go API difference between Schubfach ($\approx$342 μs/op) and Burger–Dybvig ($\approx$581 μs/op) is approximately 239 μs per canonicalization. A service verifying 1,000 signed JSON documents per second at this payload size would reclaim roughly 0.24 seconds of CPU time per second—freeing approximately 24% of a core compared to the Burger–Dybvig path.

Real-world JSON canonicalization use cases span a range of number densities: JOSE/JWS payloads are typically string-dominant (low ν), Verifiable Credentials often contain timestamps and amounts (moderate ν), and scientific or financial data payloads can be number-heavy (high ν). For workloads with $\nu < 10\%$, the digit-generation algorithm choice has no measurable impact on end-to-end latency; the practical benefit concentrates in number-dense payloads where digit generation accounts for a meaningful fraction of total cost.

8.7 Cross-Language Interpretation

Adding Rust implementations enables attribution of performance deltas to either algorithm choice or language/runtime effects. The within-language comparisons test whether the Schubfach advantage persists independent of Go's allocator and garbage collector behavior, while same-algorithm cross-language comparisons quantify language overhead (startup and steady-state) directly.

Rust's ownership model removes the need for Go-style `sync.Pool`-based reuse patterns. For multiprecision paths, this shifts lifecycle cost from GC-mediated reuse to deterministic RAII destruction, which can materially change allocation profiles and latency variance characteristics.

Startup overhead. Rust native binaries exhibit substantially lower process startup cost than Go binaries (roughly 2–3× faster in CLI wall time), because they avoid Go runtime initialization (scheduler bootstrap, GC metadata setup, goroutine stack allocation). This amplifies the startup-dominance effect discussed in §8: for small workloads the language runtime overhead is the primary differentiator, not the digit-generation algorithm.

Algorithmic ordering is language-independent. The key finding from the cross-language design is that within-Rust comparisons replicate the within-Go ordering: Schubfach outperforms Burger–Dybvig on number-dense workloads in both languages. This provides evidence that the performance advantage derives from the fixed-width digit-generation algorithm itself (zero heap allocation, no multiprecision arithmetic), not from Go-specific runtime characteristics such as garbage collection pressure or `sync.Pool` contention.

Interaction analysis. The four implementations form a 2×2 structure (algorithm $\times$ language). While we do not perform a formal factorial interaction test (e.g., two-way ANOVA or aligned rank transform) due to measurement granularity asymmetry between harnesses, Table 7 reports a ratio-of-ratios (RoR) analysis: for each canonicalize workload, R_{Go} and R_{Rust} are the within-language Schubfach/Burger–Dybvig speedup ratios, and $\text{RoR} = R_{\text{Rust}}/R_{\text{Go}}$.

RoR > 1 indicates that the Schubfach advantage is amplified in Rust. On number-dense

Table 7: Algorithm × language interaction analysis (API canonicalize mode).

Workload	R_{Go}	R_{Rust}	RoR
array-2048	1.57×	2.51×	1.60
array-256	1.56×	2.20×	1.41
canonical-minimal	1.54×	2.10×	1.36
control-escapes	1.06×	1.02×	0.97
deep-64	1.00×	1.00×	1.00
deep	1.00×	1.05×	1.05
escaped-key-order	1.60×	2.49×	1.56
large	1.72×	3.39×	1.98
long-string	1.01×	1.00×	1.00
medium	1.69×	2.97×	1.76
mixed-prod	1.31×	1.83×	1.40
nested-mixed	1.41×	1.92×	1.36
number-heavy	1.56×	8.27×	5.30
numeric-boundary	1.49×	9.32×	6.24
rfc-key-sorting	1.02×	1.01×	0.99
small	1.34×	1.75×	1.31
surrogate-pair	1.01×	1.00×	0.99
unicode	1.00×	1.03×	1.02
verify-whitespace	1.71×	2.90×	1.70

R = Schubfach/Burger–Dybvig speedup ratio. RoR = $R_{\text{Rust}}/R_{\text{Go}}$. RoR > 1 indicates Schubfach advantage is larger in Rust.

workloads, RoR consistently exceeds 1 (typically 1.3–2.0×), reflecting the larger allocation cost differential in Rust where the custom `BigUint` lacks Go's `sync.Pool` amortization. The sole exception is `escaped-key-order` (RoR = 0.97), a very small workload (18 bytes, 3 values) where non-numeric overhead dominates in both languages, leaving little room for differential amplification. This is consistent with the mechanistic explanation: both languages show the algorithmic advantage, but Rust amplifies it because its multiprecision path has higher per-operation allocation cost. Readers should interpret the RoR as descriptive evidence of interaction magnitude, not a formal statistical test of interaction significance.

Cross-language comparison caveats. Same-algorithm cross-language API comparisons (e.g., schubfach:go-vs-rs) should be interpreted carefully. Go's `testing.B` averages over many inner iterations per sample, producing low-variance measurements, while the Rust harness times individual function calls, producing higher-variance samples. Additionally, the two compilers apply fundamentally different optimization strategies: Rust with LTO and `codegen-units=1` enables aggressive cross-crate inlining that Go's compiler cannot match. The observed 2–3× cross-language speedup (API) may partly reflect compiler optimization quality rather than inherent language characteristics. These comparisons primarily serve to confirm directional consistency of the algorithmic ordering, not to quantify precise language overhead.

Allocation instrumentation. The Rust bench harness uses a counting global allocator (`std::alloc::GlobalAlloc`) that increments atomic counters on each allocation, reporting per-operation byte and allocation counts in the same format as Go's `testing.B`. This confirms the mechanistic explanation: Rust-Schubfach allocates minimally (e.g., 1,036 B/op, 24 allocs/op on `numeric-boundary`) while Rust-Burger–Dybvig allocates substantially more (25,204 B/op, 470 allocs/op on the same workload) due to `BigUint` multiprecision arithmetic. The 24.3× allocation byte ratio directly explains the 9.3× speedup: allocation and initialization cost dominates the multiprecision path.

Unicode workload anomaly. The `unicode` workload is the only API-level case where Go outperforms Rust for both algorithms: schubfach:go-vs-rs shows Go 1.08× faster, and bd:go-vs-rs shows Go 1.27× faster. This reverses the general 2–4× Rust advantage observed on all other workloads. A likely explanation is that Go's `unicode/utf8` package benefits from tight inlining of short-string fast paths in the Go compiler, whereas Rust's UTF-8 validation and string escaping may involve additional bounds checks or iterator overhead for the particular character distributions in this workload. The effect is small (both within measurement noise for the within-language comparisons, where both Go implementations and both Rust implementations use the same string-processing code) and does not affect the algorithmic conclusions.

9 Threats to Validity

9.1 Internal Validity

Warmup validation. CLI benchmarks discard 3 warmup runs. Rust API benchmarks discard 3 warmup iterations per workload to stabilize instruction caches, branch predictors, and lazily-initialized data structures (specifically, the Rust Burger–Dybvig implementation's

`OnceLock`-based power-of-ten cache). Go API benchmarks benefit from `testing.B`'s built-in warmup. Per-iteration timing data (Table 6) confirms low coefficient of variation (<5%) across measured iterations, indicating stable steady-state behavior after warmup. Figure 4 shows 20-iteration warmup validation data for a representative workload.

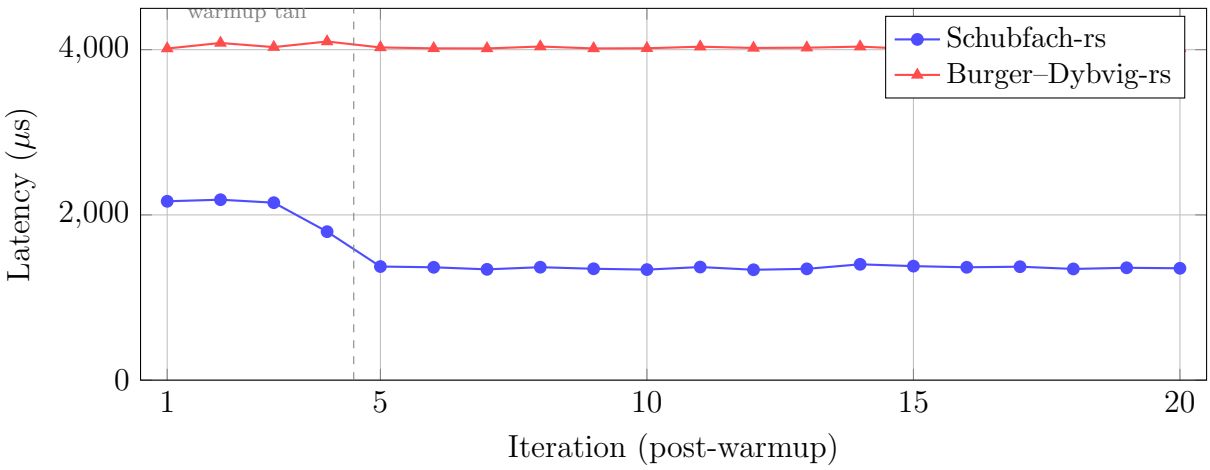

Figure 4: Per-iteration latency for `array-2048` (Rust API canonicalize, n=20 measured iterations after 3 discarded warmup iterations). `array-2048` is the largest number-dense workload in the standard set (98,901 bytes, ν=40%); if warmup stabilizes here, smaller workloads will stabilize at least as fast. Schubfach-rs shows residual cache stabilization in iterations 1–4; Burger–Dybvig-rs is stable from iteration 1. Both reach steady state by iteration 5, confirming that 3 warmup iterations are sufficient and any residual warmup effect in the first few measured iterations does not bias the within-language comparison (both implementations share the same harness).

Sequential run autocorrelation. CLI benchmarks run sequentially on the same machine; consecutive runs share CPU thermal state, cache warmth, and potentially TDP-limited frequency. The randomized task order (fixed seed) mitigates systematic ordering effects, but thermal coupling between adjacent runs creates weak positive autocorrelation that may slightly inflate the effective sample size. Permutation tests are robust to mild non-stationarity but assume exchangeability under the null hypothesis; this assumption is approximately but not exactly satisfied.

CPU core affinity. CLI benchmarks ran without CPU pinning on heterogeneous P-core/E-core topology (8 P-cores at 3.6–5.0 GHz, 4 E-cores at 2.7–3.8 GHz). Process migration may inflate variance for small workloads near the noise floor; an E-core placement could slow a single run by 30–40%. The high MDEs observed for small Go CLI workloads (14–24%) are consistent with this variance source. API benchmarks (in-process) are less susceptible.

API measurement granularity. Go API benchmarks (`testing.B`) auto-calibrate the number of inner loop iterations per sample, producing averaged low-variance samples. Rust API benchmarks time individual function calls (N=1 per sample), producing inherently higher-variance samples. This asymmetry affects cross-language comparisons but not within-language comparisons. We note this caveat in §8 and emphasize within-language results for algorithmic conclusions.

sync.Pool mitigation. The Go Burger–Dybvig implementation uses `sync.Pool` to reuse big-integer buffers, reducing allocation cost. Without pooling, the performance gap would be wider. This reflects realistic production usage but may understate the theoretical algorithmic advantage of fixed-width approaches.

9.2 Construct Validity

Allocation instrumentation. Rust allocation data is measured via a counting global allocator (`std::alloc::GlobalAlloc`) that intercepts all heap allocations and reports per-operation byte and count totals. This includes allocations from the benchmark infrastructure itself (e.g., output buffer construction), not solely the digit-generation path. Go's `testing.B` has the same characteristic. Within-language Δallocs comparisons are valid because both implementations share the same infrastructure overhead.

Compiler optimization asymmetry. Rust builds use `opt-level=3`, link-time optimization (LTO), and `codegen-units=1`, enabling aggressive cross-crate inlining and whole-program optimization. Go's compiler does not support LTO and has a less aggressive inlining heuristic. The observed cross-language speedups (2–4$\times$ in API benchmarks) may partly reflect differences in compiler optimization quality rather than intrinsic language characteristics. Within-language comparisons are unaffected since both implementations in each pair share the same compiler pipeline.

Memory allocator differences. Go uses a tcmalloc-derived allocator integrated with its garbage collector. Rust uses the system allocator (typically glibc malloc on Linux). For the multiprecision (Burger–Dybvig) path, which performs heap allocations for big-integer arithmetic, allocator performance differences could affect cross-language comparisons. Within-language comparisons are unaffected since both implementations share the same allocator.

Multiprecision implementation quality. The Rust Burger–Dybvig implementation uses a purpose-built `BigUint` with schoolbook arithmetic (approximately 350 lines), not an optimized library such as GMP or Go's `math/big`. Go's `math/big.Int` benefits from assembly-accelerated inner loops on x86-64, potentially giving it lower per-operation cost than our schoolbook Rust implementation. This means the absolute Rust Burger–Dybvig timings may be higher than they would be with a production-grade multiprecision library, and within-Rust speedup ratios correspondingly larger. Within-language comparisons still isolate the algorithmic difference (both implementations share the same allocator and compiler), but readers should note that the Rust Burger–Dybvig absolute performance reflects a research-grade bigint.

Go toolchain version. Benchmarks use Go 1.25.6. Since neither implementation uses `strconv.FormatFloat`, Go 1.26's adoption of Dragonbox for `strconv` does not affect the digit-generation paths under test. The Go compiler version affects only runtime infrastructure (allocator, scheduler, garbage collector), which is shared identically between both implementations within each pair.

Oracle SHA-256 circularity. SHA-256 checksums verify vector integrity but cannot verify oracle correctness. Independent verification against a second JS engine (SpiderMonkey, JavaScriptCore) is noted as future work.

9.3 External Validity

Language specificity (mitigated). This threat is substantially mitigated by the 2×2 cross-language comparison matrix: both algorithms are implemented in Go and Rust, and the Schubfach performance ordering holds in both languages. Within-Rust comparisons replicate the within-Go algorithmic advantage, confirming that the result is not an artifact of Go's garbage collector or allocation subsystem. Residual external-validity risk remains for additional runtimes (e.g., JVM/generational-GC and unmanaged C implementations) that are not yet benchmarked.

Single-architecture performance data. Performance benchmarks run on a single x86-64 machine. Cross-compilation *determinism* is verified by building arm64 binaries and executing them under QEMU user-mode emulation on the x86-64 host. Because QEMU user-mode translates system calls but executes floating-point operations on the host CPU, this verifies Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 *performance* benchmarks are deferred due to lack of bare-metal hardware. Future work should include bare-metal arm64 benchmarks on Graviton, Apple Silicon, or Ampere hardware.

Single JavaScript engine oracle. Oracle vectors are generated from V8 (via Node.js). While ECMA-262 `Number::toString` semantics are normative and engine-independent, our vectors are verified against only one engine and one version (V8 12.9.202.28). Cross-version stability is expected from the normative spec but not independently verified. This is mitigated by dense boundary coverage: 286,362 vectors including all subnormals up to 2^{12}, all binary64 powers of two, subnormal/normal boundary neighborhoods, RFC 8785 Appendix B examples with neighbors, and pseudorandom fill. Multi-engine validation (SpiderMonkey, JavaScriptCore) is a concrete item for future work that would strengthen oracle independence.

Workload representativeness. Benchmark workloads are synthetic, designed to bracket the performance space between number-heavy and string-dominant extremes. Real-world JSON payloads may have different number density, nesting depth, and key length distributions. However, the control group design (number-only and string-only workloads) bounds the expected range.

Endianness. Both x86-64 and arm64 targets are little-endian; big-endian behavior is untested. While the canonicalization algorithms do not depend on byte order (they operate on IEEE 754 binary64 values via language-level primitives), edge cases in binary representation parsing could theoretically differ on big-endian platforms.

10 Conclusion

We presented the first empirical comparison of multiprecision (Burger–Dybvig) and fixed-width (Schubfach) digit generation inside a RFC 8785/ECMA-262-constrained JSON canonicalization pipeline, with cross-language validation in Go and Rust to separate algorithmic effects from language/runtime characteristics. While prior work has compared float-to-string algorithms in isolation (e.g., Adams's comparison of Ryū against Grisu and Errol), our study is the first to evaluate these algorithms within a specification-constrained canonicalization

pipeline using conformance-gated methodology.

Key findings:

- Schubfach achieves 1.3–1.7× speedup on number-dense workloads ($\nu(D) \geq 40\%$) in Go API benchmarks due to zero heap allocation; the advantage replicates at 1.7–3.4× in Rust, confirming it is algorithmic. On a small number of API-level non-number workloads, Burger–Dybvig is marginally faster.
- In end-to-end Go CLI measurement, 12 of 38 comparisons reach BH-corrected significance with speedups of 1.2–1.5×.
- The advantage narrows to <5% on string-dominant and deeply-nested inputs.
- Go implementations produce byte-identical output under cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host). Rust implementations were not cross-compiled in this study.
- Preliminary profiling suggests digit generation accounts for approximately 15–40% of total canonicalization cost depending on number density.

The benchmark infrastructure, 286,362 oracle vectors, and all raw data are released as a reproducibility artifact. Our findings align with the broader ecosystem direction: Go 1.26 adopted the Dragonbox fixed-width algorithm for `strconv`, corroborating our evidence that fixed-width approaches offer meaningful performance advantages in float-to-string formatting.

Future Work

Bare-metal arm64 performance benchmarks, additional language implementations (C and JVM) to extend the comparison matrix to unmanaged and managed-GC runtimes, multi-engine oracle validation (SpiderMonkey, JavaScriptCore), Rust cross-compilation verification, and workload characterization from production JSON corpora.

Data Availability Statement

All data, software, and oracle vectors are freely available online. The benchmark infrastructure, all four implementations, workload manifests, and oracle generation scripts are available at the accompanying repository.[2] Oracle vector checksums (SHA-256) are documented in `PROVENANCE.json` files accompanying each implementation.

Author Contributions

Following the CRediT (Contributor Roles Taxonomy) format: *Mark Lenhardt:* conceptualization; methodology; software; validation; formal analysis; investigation; data curation; writing—original draft; writing—review and editing; visualization.

[2]Repository URL and Zenodo DOI to be inserted for camera-ready version.

References

[1] Ulf Adams. Ryū: Fast float-to-string conversion. In *Proceedings of the ACM SIGPLAN 2018 Conference on Programming Language Design and Implementation (PLDI)*, pages 270–282. ACM, 2018.

[2] Carsten Bormann and Paul Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, 2020.

[3] Robert G. Burger and R. Kent Dybvig. Printing floating-point numbers quickly and accurately. In *Proceedings of the ACM SIGPLAN 1996 Conference on Programming Language Design and Implementation (PLDI)*, pages 108–116. ACM, 1996.

[4] Jaël Champagne Gareau and Daniel Lemire. Converting binary floating-point numbers to shortest decimal strings: An experimental review. *Software: Practice and Experience*, 2026. Published online January 2026.

[5] Ecma International. ECMA-262: ECMAScript Language Specification, 2024.

[6] David M. Gay. Correctly rounded binary-decimal and decimal-binary conversions, 1990. AT&T Bell Laboratories Numerical Analysis Manuscript 90-10.

[7] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 57–76. ACM, 2007.

[8] Raffaello Giulietti. The schubfach way to render doubles, 2022. Technical report; reference implementation in OpenJDK (`https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/jdk/internal/math/DoubleToDecimal.java`).

[9] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Number 754-2019. IEEE, 2019.

[10] Junekey Jeon. Dragonbox: A new floating-point binary-to-decimal conversion algorithm, 2022. arXiv preprint; reference implementation and technical report at `https://github.com/jk-jeon/dragonbox`.

[11] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. *ACM SIGPLAN Notices*, 48(11):63–74, 2013.

[12] Daniel Lemire. Number parsing at a gigabyte per second. *Software: Practice and Experience*, 51(8):1700–1727, 2021.

[13] Florian Loitsch. Printing floating-point numbers quickly and accurately with integers. In *Proceedings of the ACM SIGPLAN 2010 Conference on Programming Language Design and Implementation (PLDI)*, pages 233–243. ACM, 2010.

[14] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276. ACM, 2009.

[15] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020.

[16] W3C. RDF Dataset Canonicalization, 2024. W3C Recommendation, 21 May 2024.

A Full Benchmark Data

This appendix contains the complete benchmark results. Summary tables appear in the main text (§7); the full data are provided here for reproducibility.

Note: Rust allocation data (B/op, allocs/op) is measured via a counting global allocator that intercepts all heap allocations (see §8).

A.1 Full API Benchmarks

Table 8: API benchmark results (mean of $n = 10$ iterations).

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
go:schubfach-vs-bd	array-2048	canonicalize	2590247	4079519	1.57×	+8209	+186660
go:schubfach-vs-bd	array-256	canonicalize	302598	471588	1.56×	+1020	+24977
go:schubfach-vs-bd	canonical-minimal	canonicalize	344	530	1.54×	+0	+0
go:schubfach-vs-bd	control-escapes	canonicalize	1342	1270	1.06×	+0	+0
go:schubfach-vs-bd	deep	canonicalize	36635	36716	1.00×	+0	+0
go:schubfach-vs-bd	deep-64	canonicalize	16300	16267	1.00×	+0	+0
go:schubfach-vs-bd	escaped-key-order	canonicalize	707	1130	1.60×	+2	+41
go:schubfach-vs-bd	large	canonicalize	11956587	20515409	1.72×	+32791	+663172
go:schubfach-vs-bd	long-string	canonicalize	75922	75186	1.01×	+0	+0
go:schubfach-vs-bd	medium	canonicalize	343556	580925	1.69×	+1014	+21133
go:schubfach-vs-bd	mixed-prod	canonicalize	136262	178470	1.31×	+255	+6238
go:schubfach-vs-bd	nested-mixed	canonicalize	2185	3072	1.41×	+4	+85
go:schubfach-vs-bd	number-heavy	canonicalize	13776	21508	1.56×	+20	+831
go:schubfach-vs-bd	numeric-boundary	canonicalize	12564	18754	1.49×	+14	+706
go:schubfach-vs-bd	rfc-key-sorting	canonicalize	3003	2954	1.02×	+0	+0
go:schubfach-vs-bd	small	canonicalize	1245	1664	1.34×	+2	+50
go:schubfach-vs-bd	surrogate-pair	canonicalize	648	639	1.01×	+0	+0
go:schubfach-vs-bd	unicode	canonicalize	1555	1549	1.00×	+0	+0
go:schubfach-vs-bd	verify-whitespace	canonicalize	913	1562	1.71×	+4	+82
go:schubfach-vs-bd	canonical/array-2048	verify	2495689	4044280	1.62×	+8208	+186547
go:schubfach-vs-bd	canonical/array-256	verify	317661	483083	1.52×	+1022	+25232
go:schubfach-vs-bd	canonical/canonical-minimal	verify	358	543	1.52×	+0	+0
go:schubfach-vs-bd	canonical/control-escapes	verify	1320	1268	1.04×	+0	+0
go:schubfach-vs-bd	canonical/deep	verify	37822	38059	1.01×	+0	+0
go:schubfach-vs-bd	canonical/deep-64	verify	16715	16792	1.00×	+0	+0
go:schubfach-vs-bd	canonical/escaped-key-order	verify	686	1120	1.63×	+2	+41
go:schubfach-vs-bd	canonical/large	verify	11513704	20066364	1.74×	+32789	+662960
go:schubfach-vs-bd	canonical/long-string	verify	77633	77510	1.00×	+0	+0
go:schubfach-vs-bd	canonical/medium	verify	344438	580112	1.68×	+1015	+21371
go:schubfach-vs-bd	canonical/mixed-prod	verify	135662	176984	1.30×	+255	+6325
go:schubfach-vs-bd	canonical/nested-mixed	verify	2218	3118	1.41×	+4	+86
go:schubfach-vs-bd	canonical/number-heavy	verify	13996	21584	1.54×	+20	+834
go:schubfach-vs-bd	canonical/numeric-boundary	verify	12840	18852	1.47×	+14	+709
go:schubfach-vs-bd	canonical/rfc-key-sorting	verify	2884	2846	1.01×	+0	+0
go:schubfach-vs-bd	canonical/small	verify	1230	1659	1.35×	+2	+51
go:schubfach-vs-bd	canonical/surrogate-pair	verify	660	655	1.01×	+0	+0
go:schubfach-vs-bd	canonical/unicode	verify	1554	1550	1.00×	+0	+0
go:schubfach-vs-bd	canonical/verify-whitespace	verify	939	1594	1.70×	+4	+82
go:schubfach-vs-bd	noncanonical/control-escapes	verify	1328	1267	1.05×	+0	+0
go:schubfach-vs-bd	noncanonical/escaped-key-order	verify	707	1134	1.60×	+2	+41

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
go:schubfach-vs-bd	noncanonical/large	verify	11993194	20682335	1.72×	+32793	+663520
go:schubfach-vs-bd	noncanonical/medium	verify	340992	572859	1.68×	+1014	+21136
go:schubfach-vs-bd	noncanonical/mixed-prod	verify	136287	178028	1.31×	+255	+6227
go:schubfach-vs-bd	noncanonical/nested-mixed	verify	2188	3085	1.41×	+4	+85
go:schubfach-vs-bd	noncanonical/number-heavy	verify	13788	21369	1.55×	+20	+830
go:schubfach-vs-bd	noncanonical/numeric-boundary	verify	12604	18770	1.49×	+14	+706
go:schubfach-vs-bd	noncanonical/rfc-key-sorting	verify	2990	2982	1.00×	+0	+0
go:schubfach-vs-bd	noncanonical/small	verify	1241	1670	1.35×	+2	+50
go:schubfach-vs-bd	noncanonical/surrogate-pair	verify	649	644	1.01×	+0	+0
go:schubfach-vs-bd	noncanonical/unicode	verify	1551	1554	1.00×	+0	+0
go:schubfach-vs-bd	noncanonical/verify-whitespace	verify	910	1568	1.72×	+4	+82
rs:schubfach-vs-bd	array-2048	canonicalize	1361726	3424060	2.51×	+141429	+1814032
rs:schubfach-vs-bd	array-256	canonicalize	171544	378233	2.20×	+14906	+189584
rs:schubfach-vs-bd	canonical-minimal	canonicalize	211	443	2.10×	+16	+200
rs:schubfach-vs-bd	control-escapes	canonicalize	625	611	1.02×	+0	+0
rs:schubfach-vs-bd	deep	canonicalize	30335	28940	1.05×	+0	+0
rs:schubfach-vs-bd	deep-64	canonicalize	10921	10925	1.00×	+0	+0
rs:schubfach-vs-bd	escaped-key-order	canonicalize	327	816	2.49×	+33	+408
rs:schubfach-vs-bd	large	canonicalize	5782747	19608594	3.39×	+926334	+12054624
rs:schubfach-vs-bd	long-string	canonicalize	17198	17116	1.00×	+0	+0
rs:schubfach-vs-bd	medium	canonicalize	174362	517769	2.97×	+24337	+314864
rs:schubfach-vs-bd	mixed-prod	canonicalize	66094	120970	1.83×	+3941	+49904
rs:schubfach-vs-bd	nested-mixed	canonicalize	1020	1956	1.92×	+66	+816
rs:schubfach-vs-bd	number-heavy	canonicalize	1417	11729	8.27×	+594	+26080
rs:schubfach-vs-bd	numeric-boundary	canonicalize	954	8888	9.32×	+446	+24168
rs:schubfach-vs-bd	rfc-key-sorting	canonicalize	1315	1307	1.01×	+0	+0
rs:schubfach-vs-bd	small	canonicalize	632	1104	1.75×	+33	+408
rs:schubfach-vs-bd	surrogate-pair	canonicalize	311	310	1.00×	+0	+0
rs:schubfach-vs-bd	unicode	canonicalize	1478	1518	1.03×	+0	+0
rs:schubfach-vs-bd	verify-whitespace	canonicalize	400	1162	2.90×	+50	+616
rs:schubfach-vs-bd	canonical/array-2048	verify	1354255	3433183	2.54×	+141429	+1814032
rs:schubfach-vs-bd	canonical/array-256	verify	169071	379286	2.24×	+14906	+189584
rs:schubfach-vs-bd	canonical/canonical-minimal	verify	205	436	2.13×	+16	+200
rs:schubfach-vs-bd	canonical/control-escapes	verify	632	622	1.01×	+0	+0
rs:schubfach-vs-bd	canonical/deep	verify	28835	28169	1.02×	+0	+0
rs:schubfach-vs-bd	canonical/deep-64	verify	10979	10731	1.02×	+0	+0
rs:schubfach-vs-bd	canonical/escaped-key-order	verify	330	820	2.48×	+33	+408
rs:schubfach-vs-bd	canonical/large	verify	5909871	19345720	3.27×	+926334	+12054624
rs:schubfach-vs-bd	canonical/long-string	verify	17097	16950	1.01×	+0	+0
rs:schubfach-vs-bd	canonical/medium	verify	198814	524078	2.64×	+24337	+314864
rs:schubfach-vs-bd	canonical/mixed-prod	verify	65071	121484	1.87×	+3941	+49904
rs:schubfach-vs-bd	canonical/nested-mixed	verify	1001	1970	1.97×	+66	+816
rs:schubfach-vs-bd	canonical/number-heavy	verify	1383	11621	8.40×	+594	+26080
rs:schubfach-vs-bd	canonical/numeric-boundary	verify	950	8933	9.40×	+446	+24168
rs:schubfach-vs-bd	canonical/rfc-key-sorting	verify	1353	1324	1.02×	+0	+0
rs:schubfach-vs-bd	canonical/small	verify	665	1138	1.71×	+33	+408

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
rs:schubfach-vs-bd	canonical/surrogate-pair	verify	312	311	1.00×	+0	+0
rs:schubfach-vs-bd	canonical/unicode	verify	1522	1703	1.12×	+0	+0
rs:schubfach-vs-bd	canonical/verify-whitespace	verify	398	1162	2.92×	+50	+616
schubfach:go-vs-rs	array-2048	canonicalize	2590247	1361726	1.90×	-12253	-3191525
schubfach:go-vs-rs	array-256	canonicalize	302598	171544	1.76×	-1509	-387568
schubfach:go-vs-rs	canonical-minimal	canonicalize	344	211	1.63×	+0	-105
schubfach:go-vs-rs	control-escapes	canonicalize	1342	625	2.15×	-8	-1495
schubfach:go-vs-rs	deep	canonicalize	36635	30335	1.21×	-141	-3696
schubfach:go-vs-rs	deep-64	canonicalize	16300	10921	1.49×	-65	-1704
schubfach:go-vs-rs	escaped-key-order	canonicalize	707	327	2.16×	-2	-707
schubfach:go-vs-rs	large	canonicalize	11956587	5782747	2.07×	-57301	-13736566
schubfach:go-vs-rs	long-string	canonicalize	75922	17198	4.41×	-4	-2765
schubfach:go-vs-rs	medium	canonicalize	343556	174362	1.97×	-1758	-396434
schubfach:go-vs-rs	mixed-prod	canonicalize	136262	66094	2.06×	-700	-165818
schubfach:go-vs-rs	nested-mixed	canonicalize	2185	1020	2.14×	-8	-2352
schubfach:go-vs-rs	number-heavy	canonicalize	13776	1417	9.72×	-23	-3036
schubfach:go-vs-rs	numeric-boundary	canonicalize	12564	954	13.17×	-14	-2172
schubfach:go-vs-rs	rfc-key-sorting	canonicalize	3003	1315	2.28×	-20	-2569
schubfach:go-vs-rs	small	canonicalize	1245	632	1.97×	-3	-1375
schubfach:go-vs-rs	surrogate-pair	canonicalize	648	311	2.08×	-4	-713
schubfach:go-vs-rs	unicode	canonicalize	1555	1478	1.05×	-6	-1419
schubfach:go-vs-rs	verify-whitespace	canonicalize	913	400	2.28×	-4	-1002
schubfach:go-vs-rs	canonical/array-2048	verify	2495689	1354255	1.84×	-12253	-3191524
schubfach:go-vs-rs	canonical/array-256	verify	317661	169071	1.88×	-1509	-387570
schubfach:go-vs-rs	canonical/canonical-minimal	verify	358	205	1.75×	+0	-105
schubfach:go-vs-rs	canonical/control-escapes	verify	1320	632	2.09×	-8	-1495
schubfach:go-vs-rs	canonical/deep	verify	37822	28835	1.31×	-141	-3696
schubfach:go-vs-rs	canonical/deep-64	verify	16715	10979	1.52×	-65	-1704
schubfach:go-vs-rs	canonical/escaped-key-order	verify	686	330	2.08×	-2	-735
schubfach:go-vs-rs	canonical/large	verify	11513704	5909871	1.95×	-57301	-13737454
schubfach:go-vs-rs	canonical/long-string	verify	77633	17097	4.54×	-4	-2765
schubfach:go-vs-rs	canonical/medium	verify	344438	198814	1.73×	-1758	-396471
schubfach:go-vs-rs	canonical/mixed-prod	verify	135662	65071	2.08×	-700	-165845
schubfach:go-vs-rs	canonical/nested-mixed	verify	2218	1001	2.22×	-8	-2352
schubfach:go-vs-rs	canonical/number-heavy	verify	13996	1383	10.12×	-23	-3040
schubfach:go-vs-rs	canonical/numeric-boundary	verify	12840	950	13.51×	-14	-2172
schubfach:go-vs-rs	canonical/rfc-key-sorting	verify	2884	1353	2.13×	-15	-2575
schubfach:go-vs-rs	canonical/small	verify	1230	665	1.85×	-3	-1378
schubfach:go-vs-rs	canonical/surrogate-pair	verify	660	312	2.12×	-4	-713
schubfach:go-vs-rs	canonical/unicode	verify	1554	1522	1.02×	-6	-1405
schubfach:go-vs-rs	canonical/verify-whitespace	verify	939	398	2.36×	-4	-1009
bd:go-vs-rs	array-2048	canonicalize	4079519	3424060	1.19×	+120967	-1564154
bd:go-vs-rs	array-256	canonicalize	471588	378233	1.25×	+12377	-222961
bd:go-vs-rs	canonical-minimal	canonicalize	530	443	1.20×	+16	+95
bd:go-vs-rs	control-escapes	canonicalize	1270	611	2.08×	-8	-1495
bd:go-vs-rs	deep	canonicalize	36716	28940	1.27×	-141	-3696

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
bd:go-vs-rs	deep-64	canonicalize	16267	10925	1.49×	-65	-1704
bd:go-vs-rs	escaped-key-order	canonicalize	1130	816	1.38×	+29	-340
bd:go-vs-rs	large	canonicalize	20515409	19608594	1.05×	+836242	-2345114
bd:go-vs-rs	long-string	canonicalize	75186	17116	4.39×	-4	-2765
bd:go-vs-rs	medium	canonicalize	580925	517769	1.12×	+21565	-102703
bd:go-vs-rs	mixed-prod	canonicalize	178470	120970	1.48×	+2986	-122152
bd:go-vs-rs	nested-mixed	canonicalize	3072	1956	1.57×	+54	-1621
bd:go-vs-rs	number-heavy	canonicalize	21508	11729	1.83×	+551	+22213
bd:go-vs-rs	numeric-boundary	canonicalize	18754	8888	2.11×	+418	+21290
bd:go-vs-rs	rfc-key-sorting	canonicalize	2954	1307	2.26×	-20	-2569
bd:go-vs-rs	small	canonicalize	1664	1104	1.51×	+28	-1017
bd:go-vs-rs	surrogate-pair	canonicalize	639	310	2.06×	-4	-713
bd:go-vs-rs	unicode	canonicalize	1549	1518	1.02×	-6	-1419
bd:go-vs-rs	verify-whitespace	canonicalize	1562	1162	1.34×	+42	-468
bd:go-vs-rs	canonical/array-2048	verify	4044280	3433183	1.18×	+120968	-1564038
bd:go-vs-rs	canonical/array-256	verify	483083	379286	1.27×	+12375	-223217
bd:go-vs-rs	canonical/canonical-minimal	verify	543	436	1.24×	+16	+95
bd:go-vs-rs	canonical/control-escapes	verify	1268	622	2.04×	-8	-1495
bd:go-vs-rs	canonical/deep	verify	38059	28169	1.35×	-141	-3696
bd:go-vs-rs	canonical/deep-64	verify	16792	10731	1.56×	-65	-1704
bd:go-vs-rs	canonical/escaped-key-order	verify	1120	820	1.37×	+29	-368
bd:go-vs-rs	canonical/large	verify	20066364	19345720	1.04×	+836244	-2345790
bd:go-vs-rs	canonical/long-string	verify	77510	16950	4.57×	-4	-2765
bd:go-vs-rs	canonical/medium	verify	580112	524078	1.11×	+21564	-102978
bd:go-vs-rs	canonical/mixed-prod	verify	176984	121484	1.46×	+2986	-122266
bd:go-vs-rs	canonical/nested-mixed	verify	3118	1970	1.58×	+54	-1622
bd:go-vs-rs	canonical/number-heavy	verify	21584	11621	1.86×	+551	+22206
bd:go-vs-rs	canonical/numeric-boundary	verify	18852	8933	2.11×	+418	+21287
bd:go-vs-rs	canonical/rfc-key-sorting	verify	2846	1324	2.15×	-15	-2575
bd:go-vs-rs	canonical/small	verify	1659	1138	1.46×	+28	-1021
bd:go-vs-rs	canonical/surrogate-pair	verify	655	311	2.11×	-4	-713
bd:go-vs-rs	canonical/unicode	verify	1550	1703	1.10×	-6	-1405
bd:go-vs-rs	canonical/verify-whitespace	verify	1594	1162	1.37×	+42	-475

B/op = bytes allocated per operation. allocs/op = heap allocations per operation.
Δ columns show Impl B − Impl A for each pair.

A.2 Full CLI Benchmarks

Table 9: CLI benchmark results (end-to-end, $n = 15$).

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
go:schubfach-vs-bd	array-2048	canonicalize	5.388	7.210	1.34×	schubfach
go:schubfach-vs-bd	array-2048	verify	5.678	7.445	1.31×	schubfach
go:schubfach-vs-bd	array-256	canonicalize	1.980	2.342	1.18×	schubfach
go:schubfach-vs-bd	array-256	verify	1.958	2.223	1.14×	schubfach
go:schubfach-vs-bd	canonical-minimal	canonicalize	1.643	1.594	1.03×	json-canon
go:schubfach-vs-bd	canonical-minimal	verify	1.470	1.526	1.04×	schubfach
go:schubfach-vs-bd	control-escapes	canonicalize	1.573	1.612	1.02×	schubfach
go:schubfach-vs-bd	control-escapes	verify	1.539	1.629	1.06×	schubfach
go:schubfach-vs-bd	deep	canonicalize	1.658	1.599	1.04×	json-canon
go:schubfach-vs-bd	deep	verify	1.770	1.790	1.01×	schubfach
go:schubfach-vs-bd	deep-64	canonicalize	1.562	1.581	1.01×	schubfach
go:schubfach-vs-bd	deep-64	verify	1.594	1.863	1.17×	schubfach
go:schubfach-vs-bd	escaped-key-order	canonicalize	1.479	1.618	1.09×	schubfach
go:schubfach-vs-bd	escaped-key-order	verify	1.450	1.646	1.14×	schubfach
go:schubfach-vs-bd	large	canonicalize	16.246	24.723	1.52×	schubfach
go:schubfach-vs-bd	large	verify	15.821	24.700	1.56×	schubfach
go:schubfach-vs-bd	long-string	canonicalize	1.565	1.738	1.11×	schubfach
go:schubfach-vs-bd	long-string	verify	1.863	1.814	1.03×	json-canon
go:schubfach-vs-bd	medium	canonicalize	2.081	2.623	1.26×	schubfach
go:schubfach-vs-bd	medium	verify	2.007	2.321	1.16×	schubfach
go:schubfach-vs-bd	mixed-prod	canonicalize	1.731	1.835	1.06×	schubfach
go:schubfach-vs-bd	mixed-prod	verify	1.833	2.036	1.11×	schubfach
go:schubfach-vs-bd	nested-mixed	canonicalize	1.407	1.660	1.18×	schubfach
go:schubfach-vs-bd	nested-mixed	verify	1.490	1.703	1.14×	schubfach
go:schubfach-vs-bd	number-heavy	canonicalize	1.476	1.689	1.14×	schubfach
go:schubfach-vs-bd	number-heavy	verify	1.534	1.735	1.13×	schubfach
go:schubfach-vs-bd	numeric-boundary	canonicalize	1.498	1.720	1.15×	schubfach
go:schubfach-vs-bd	numeric-boundary	verify	1.486	1.590	1.07×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	canonicalize	1.527	1.542	1.01×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	verify	1.544	1.510	1.02×	json-canon
go:schubfach-vs-bd	small	canonicalize	1.447	1.766	1.22×	schubfach
go:schubfach-vs-bd	small	verify	1.446	1.553	1.07×	schubfach
go:schubfach-vs-bd	surrogate-pair	canonicalize	1.513	1.587	1.05×	schubfach
go:schubfach-vs-bd	surrogate-pair	verify	1.489	1.564	1.05×	schubfach
go:schubfach-vs-bd	unicode	canonicalize	1.521	1.688	1.11×	schubfach
go:schubfach-vs-bd	unicode	verify	1.413	1.607	1.14×	schubfach
go:schubfach-vs-bd	verify-whitespace	canonicalize	1.487	1.631	1.10×	schubfach
go:schubfach-vs-bd	verify-whitespace	verify	1.324	1.645	1.24×	schubfach
rs:schubfach-vs-bd	array-2048	canonicalize	2.424	4.487	1.85×	schubfach-rs
rs:schubfach-vs-bd	array-2048	verify	2.594	4.245	1.64×	schubfach-rs
rs:schubfach-vs-bd	array-256	canonicalize	0.848	1.214	1.43×	schubfach-rs
rs:schubfach-vs-bd	array-256	verify	0.815	1.159	1.42×	schubfach-rs
rs:schubfach-vs-bd	canonical-minimal	canonicalize	0.571	0.640	1.12×	schubfach-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
rs:schubfach-vs-bd	canonical-minimal	verify	0.539	0.613	1.14×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	canonicalize	0.543	0.621	1.14×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	verify	0.563	0.630	1.12×	schubfach-rs
rs:schubfach-vs-bd	deep	canonicalize	0.637	0.712	1.12×	schubfach-rs
rs:schubfach-vs-bd	deep	verify	0.751	0.734	1.02×	json-canon-rs
rs:schubfach-vs-bd	deep-64	canonicalize	0.581	0.646	1.11×	schubfach-rs
rs:schubfach-vs-bd	deep-64	verify	0.591	0.645	1.09×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	canonicalize	0.577	0.675	1.17×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	verify	0.569	0.720	1.27×	schubfach-rs
rs:schubfach-vs-bd	large	canonicalize	7.608	18.530	2.44×	schubfach-rs
rs:schubfach-vs-bd	large	verify	7.489	18.519	2.47×	schubfach-rs
rs:schubfach-vs-bd	long-string	canonicalize	0.622	0.679	1.09×	schubfach-rs
rs:schubfach-vs-bd	long-string	verify	0.613	0.673	1.10×	schubfach-rs
rs:schubfach-vs-bd	medium	canonicalize	0.861	1.426	1.66×	schubfach-rs
rs:schubfach-vs-bd	medium	verify	0.765	1.439	1.88×	schubfach-rs
rs:schubfach-vs-bd	mixed-prod	canonicalize	0.680	0.910	1.34×	schubfach-rs
rs:schubfach-vs-bd	mixed-prod	verify	0.669	0.865	1.29×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	canonicalize	0.569	0.712	1.25×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	verify	0.553	0.729	1.32×	schubfach-rs
rs:schubfach-vs-bd	number-heavy	canonicalize	0.538	0.785	1.46×	schubfach-rs
rs:schubfach-vs-bd	number-heavy	verify	0.546	0.769	1.41×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	canonicalize	0.549	0.702	1.28×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	verify	0.587	0.727	1.24×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	canonicalize	0.567	0.641	1.13×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	verify	0.576	0.616	1.07×	schubfach-rs
rs:schubfach-vs-bd	small	canonicalize	0.571	0.697	1.22×	schubfach-rs
rs:schubfach-vs-bd	small	verify	0.573	0.660	1.15×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	canonicalize	0.551	0.599	1.09×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	verify	0.559	0.642	1.15×	schubfach-rs
rs:schubfach-vs-bd	unicode	canonicalize	0.563	0.605	1.07×	schubfach-rs
rs:schubfach-vs-bd	unicode	verify	0.569	0.619	1.09×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	canonicalize	0.561	0.749	1.34×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	verify	0.585	0.698	1.19×	schubfach-rs
schubfach:go-vs-rs	array-2048	canonicalize	5.388	2.424	2.22×	schubfach-rs
schubfach:go-vs-rs	array-2048	verify	5.678	2.594	2.19×	schubfach-rs
schubfach:go-vs-rs	array-256	canonicalize	1.980	0.848	2.33×	schubfach-rs
schubfach:go-vs-rs	array-256	verify	1.958	0.815	2.40×	schubfach-rs
schubfach:go-vs-rs	canonical-minimal	canonicalize	1.643	0.571	2.88×	schubfach-rs
schubfach:go-vs-rs	canonical-minimal	verify	1.470	0.539	2.73×	schubfach-rs
schubfach:go-vs-rs	control-escapes	canonicalize	1.573	0.543	2.90×	schubfach-rs
schubfach:go-vs-rs	control-escapes	verify	1.539	0.563	2.73×	schubfach-rs
schubfach:go-vs-rs	deep	canonicalize	1.658	0.637	2.60×	schubfach-rs
schubfach:go-vs-rs	deep	verify	1.770	0.751	2.36×	schubfach-rs
schubfach:go-vs-rs	deep-64	canonicalize	1.562	0.581	2.69×	schubfach-rs
schubfach:go-vs-rs	deep-64	verify	1.594	0.591	2.70×	schubfach-rs
schubfach:go-vs-rs	escaped-key-order	canonicalize	1.479	0.577	2.57×	schubfach-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
schubfach:go-vs-rs	escaped-key-order	verify	1.450	0.569	2.55×	schubfach-rs
schubfach:go-vs-rs	large	canonicalize	16.246	7.608	2.14×	schubfach-rs
schubfach:go-vs-rs	large	verify	15.821	7.489	2.11×	schubfach-rs
schubfach:go-vs-rs	long-string	canonicalize	1.565	0.622	2.52×	schubfach-rs
schubfach:go-vs-rs	long-string	verify	1.863	0.613	3.04×	schubfach-rs
schubfach:go-vs-rs	medium	canonicalize	2.081	0.861	2.42×	schubfach-rs
schubfach:go-vs-rs	medium	verify	2.007	0.765	2.62×	schubfach-rs
schubfach:go-vs-rs	mixed-prod	canonicalize	1.731	0.680	2.55×	schubfach-rs
schubfach:go-vs-rs	mixed-prod	verify	1.833	0.669	2.74×	schubfach-rs
schubfach:go-vs-rs	nested-mixed	canonicalize	1.407	0.569	2.47×	schubfach-rs
schubfach:go-vs-rs	nested-mixed	verify	1.490	0.553	2.70×	schubfach-rs
schubfach:go-vs-rs	number-heavy	canonicalize	1.476	0.538	2.74×	schubfach-rs
schubfach:go-vs-rs	number-heavy	verify	1.534	0.546	2.81×	schubfach-rs
schubfach:go-vs-rs	numeric-boundary	canonicalize	1.498	0.549	2.73×	schubfach-rs
schubfach:go-vs-rs	numeric-boundary	verify	1.486	0.587	2.53×	schubfach-rs
schubfach:go-vs-rs	rfc-key-sorting	canonicalize	1.527	0.567	2.69×	schubfach-rs
schubfach:go-vs-rs	rfc-key-sorting	verify	1.544	0.576	2.68×	schubfach-rs
schubfach:go-vs-rs	small	canonicalize	1.447	0.571	2.54×	schubfach-rs
schubfach:go-vs-rs	small	verify	1.446	0.573	2.52×	schubfach-rs
schubfach:go-vs-rs	surrogate-pair	canonicalize	1.513	0.551	2.74×	schubfach-rs
schubfach:go-vs-rs	surrogate-pair	verify	1.489	0.559	2.66×	schubfach-rs
schubfach:go-vs-rs	unicode	canonicalize	1.521	0.563	2.70×	schubfach-rs
schubfach:go-vs-rs	unicode	verify	1.413	0.569	2.48×	schubfach-rs
schubfach:go-vs-rs	verify-whitespace	canonicalize	1.487	0.561	2.65×	schubfach-rs
schubfach:go-vs-rs	verify-whitespace	verify	1.324	0.585	2.26×	schubfach-rs
bd:go-vs-rs	array-2048	canonicalize	7.210	4.487	1.61×	json-canon-rs
bd:go-vs-rs	array-2048	verify	7.445	4.245	1.75×	json-canon-rs
bd:go-vs-rs	array-256	canonicalize	2.342	1.214	1.93×	json-canon-rs
bd:go-vs-rs	array-256	verify	2.223	1.159	1.92×	json-canon-rs
bd:go-vs-rs	canonical-minimal	canonicalize	1.594	0.640	2.49×	json-canon-rs
bd:go-vs-rs	canonical-minimal	verify	1.526	0.613	2.49×	json-canon-rs
bd:go-vs-rs	control-escapes	canonicalize	1.612	0.621	2.60×	json-canon-rs
bd:go-vs-rs	control-escapes	verify	1.629	0.630	2.59×	json-canon-rs
bd:go-vs-rs	deep	canonicalize	1.599	0.712	2.24×	json-canon-rs
bd:go-vs-rs	deep	verify	1.790	0.734	2.44×	json-canon-rs
bd:go-vs-rs	deep-64	canonicalize	1.581	0.646	2.45×	json-canon-rs
bd:go-vs-rs	deep-64	verify	1.863	0.645	2.89×	json-canon-rs
bd:go-vs-rs	escaped-key-order	canonicalize	1.618	0.675	2.40×	json-canon-rs
bd:go-vs-rs	escaped-key-order	verify	1.646	0.720	2.29×	json-canon-rs
bd:go-vs-rs	large	canonicalize	24.723	18.530	1.33×	json-canon-rs
bd:go-vs-rs	large	verify	24.700	18.519	1.33×	json-canon-rs
bd:go-vs-rs	long-string	canonicalize	1.738	0.679	2.56×	json-canon-rs
bd:go-vs-rs	long-string	verify	1.814	0.673	2.70×	json-canon-rs
bd:go-vs-rs	medium	canonicalize	2.623	1.426	1.84×	json-canon-rs
bd:go-vs-rs	medium	verify	2.321	1.439	1.61×	json-canon-rs
bd:go-vs-rs	mixed-prod	canonicalize	1.835	0.910	2.02×	json-canon-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
bd:go-vs-rs	mixed-prod	verify	2.036	0.865	2.35×	json-canon-rs
bd:go-vs-rs	nested-mixed	canonicalize	1.660	0.712	2.33×	json-canon-rs
bd:go-vs-rs	nested-mixed	verify	1.703	0.729	2.34×	json-canon-rs
bd:go-vs-rs	number-heavy	canonicalize	1.689	0.785	2.15×	json-canon-rs
bd:go-vs-rs	number-heavy	verify	1.735	0.769	2.26×	json-canon-rs
bd:go-vs-rs	numeric-boundary	canonicalize	1.720	0.702	2.45×	json-canon-rs
bd:go-vs-rs	numeric-boundary	verify	1.590	0.727	2.19×	json-canon-rs
bd:go-vs-rs	rfc-key-sorting	canonicalize	1.542	0.641	2.41×	json-canon-rs
bd:go-vs-rs	rfc-key-sorting	verify	1.510	0.616	2.45×	json-canon-rs
bd:go-vs-rs	small	canonicalize	1.766	0.697	2.53×	json-canon-rs
bd:go-vs-rs	small	verify	1.553	0.660	2.35×	json-canon-rs
bd:go-vs-rs	surrogate-pair	canonicalize	1.587	0.599	2.65×	json-canon-rs
bd:go-vs-rs	surrogate-pair	verify	1.564	0.642	2.44×	json-canon-rs
bd:go-vs-rs	unicode	canonicalize	1.688	0.605	2.79×	json-canon-rs
bd:go-vs-rs	unicode	verify	1.607	0.619	2.60×	json-canon-rs
bd:go-vs-rs	verify-whitespace	canonicalize	1.631	0.749	2.18×	json-canon-rs
bd:go-vs-rs	verify-whitespace	verify	1.645	0.698	2.36×	json-canon-rs

Speedup = slower/faster mean wall time. $n = 15$ repetitions, 3 warmup runs discarded.

A.3 Full Statistical Comparisons

Table 10: Statistical comparisons with Benjamini-Hochberg corrected p-values.

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
go:schubfach-vs-bd	e2e	canonicalize	array-2048	schubfach	json-canon	schubfach*	1.34×	[1.270, 1.414]	0.0002	0.0003	-3.48	
rs:schubfach-vs-bd	e2e	canonicalize	array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	1.85×	[1.668, 2.062]	0.0002	0.0003	-3.89	
schubfach:go-vs-rs	e2e	canonicalize	array-2048	schubfach	schubfach-rs	schubfach-rs*	2.22×	[2.031, 2.411]	0.0002	0.0003	7.90	
bd:go-vs-rs	e2e	canonicalize	array-2048	json-canon	json-canon-rs	json-canon-rs*	1.61×	[1.461, 1.727]	0.0002	0.0003	4.23	
go:schubfach-vs-bd	e2e	canonicalize	array-256	schubfach	json-canon	schubfach*	1.18×	[1.046, 1.326]	0.0142	0.0176	-0.99	
rs:schubfach-vs-bd	e2e	canonicalize	array-256	schubfach-rs	json-canon-rs	schubfach-rs*	1.43×	[1.319, 1.530]	0.0002	0.0003	-3.16	
schubfach:go-vs-rs	e2e	canonicalize	array-256	schubfach	schubfach-rs	schubfach-rs*	2.33×	[2.152, 2.558]	0.0002	0.0003	5.01	
bd:go-vs-rs	e2e	canonicalize	array-256	json-canon	json-canon-rs	json-canon-rs*	1.93×	[1.736, 2.148]	0.0002	0.0003	3.61	
go:schubfach-vs-bd	e2e	canonicalize	canonical-minimal	schubfach	json-canon	json-canon	1.03×	[0.929, 1.149]	0.5919	0.6254	0.20	
rs:schubfach-vs-bd	e2e	canonicalize	canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs*	1.12×	[1.042, 1.192]	0.0040	0.0051	-1.12	
schubfach:go-vs-rs	e2e	canonicalize	canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	2.88×	[2.692, 3.047]	0.0002	0.0003	7.59	
bd:go-vs-rs	e2e	canonicalize	canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	2.49×	[2.209, 2.774]	0.0002	0.0003	4.50	
go:schubfach-vs-bd	e2e	canonicalize	control-escapes	schubfach	json-canon	schubfach	1.02×	[0.907, 1.166]	0.7327	0.7615	-0.13	
rs:schubfach-vs-bd	e2e	canonicalize	control-escapes	schubfach-rs	json-canon-rs	schubfach-rs*	1.14×	[1.053, 1.223]	0.0014	0.0018	-1.28	
schubfach:go-vs-rs	e2e	canonicalize	control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.90×	[2.592, 3.164]	0.0002	0.0003	5.77	
bd:go-vs-rs	e2e	canonicalize	control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.60×	[2.315, 2.883]	0.0002	0.0003	4.15	
go:schubfach-vs-bd	e2e	canonicalize	deep-64	schubfach	json-canon	schubfach	1.01×	[0.883, 1.181]	0.8730	0.8927	-0.06	

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
rs:schubfach-vs-bd	e2e	canonicalize	deep-64	schubfach-rs	json-canon-rs	schubfach-rs*	1.11×	[1.032, 1.205]	0.0148	0.0183	-0.94	
schubfach:go-vs-rs	e2e	canonicalize	deep-64	schubfach	schubfach-rs	schubfach-rs*	2.69×	[2.401, 2.979]	0.0002	0.0003	4.65	
bd:go-vs-rs	e2e	canonicalize	deep-64	json-canon	json-canon-rs	json-canon-rs*	2.45×	[2.181, 2.789]	0.0002	0.0003	3.66	
go:schubfach-vs-bd	e2e	canonicalize	deep	schubfach	json-canon	json-canon	1.04×	[0.901, 1.186]	0.6259	0.6570	0.18	
rs:schubfach-vs-bd	e2e	canonicalize	deep	schubfach-rs	json-canon-rs	schubfach-rs*	1.12×	[1.060, 1.171]	0.0004	0.0005	-1.57	
schubfach:go-vs-rs	e2e	canonicalize	deep	schubfach	schubfach-rs	schubfach-rs*	2.60×	[2.338, 2.890]	0.0002	0.0003	4.23	
bd:go-vs-rs	e2e	canonicalize	deep	json-canon	json-canon-rs	json-canon-rs*	2.24×	[2.041, 2.487]	0.0002	0.0003	3.98	
go:schubfach-vs-bd	e2e	canonicalize	escaped-key-order	schubfach	json-canon	schubfach	1.09×	[0.947, 1.262]	0.2517	0.2800	-0.43	
rs:schubfach-vs-bd	e2e	canonicalize	escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	1.17×	[1.097, 1.248]	0.0002	0.0003	-1.66	
schubfach:go-vs-rs	e2e	canonicalize	escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.57×	[2.310, 2.870]	0.0002	0.0003	4.25	
bd:go-vs-rs	e2e	canonicalize	escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	2.40×	[2.132, 2.693]	0.0002	0.0003	3.79	
go:schubfach-vs-bd	e2e	canonicalize	large	schubfach	json-canon	schubfach*	1.52×	[1.477, 1.572]	0.0002	0.0003	-9.85	
rs:schubfach-vs-bd	e2e	canonicalize	large	schubfach-rs	json-canon-rs	schubfach-rs*	2.44×	[2.353, 2.522]	0.0002	0.0003	-21.29	
schubfach:go-vs-rs	e2e	canonicalize	large	schubfach	schubfach-rs	schubfach-rs*	2.14×	[2.050, 2.230]	0.0002	0.0003	12.06	
bd:go-vs-rs	e2e	canonicalize	large	json-canon	json-canon-rs	json-canon-rs*	1.33×	[1.305, 1.363]	0.0002	0.0003	8.84	
go:schubfach-vs-bd	e2e	canonicalize	long-string	schubfach	json-canon	schubfach	1.11×	[0.953, 1.282]	0.1824	0.2050	-0.49	
rs:schubfach-vs-bd	e2e	canonicalize	long-string	schubfach-rs	json-canon-rs	schubfach-rs*	1.09×	[1.024, 1.165]	0.0122	0.0152	-0.94	
schubfach:go-vs-rs	e2e	canonicalize	long-string	schubfach	schubfach-rs	schubfach-rs*	2.52×	[2.247, 2.814]	0.0002	0.0003	3.91	
bd:go-vs-rs	e2e	canonicalize	long-string	json-canon	json-canon-rs	json-canon-rs*	2.56×	[2.283, 2.852]	0.0002	0.0003	4.07	
go:schubfach-vs-bd	e2e	canonicalize	medium	schubfach	json-canon	schubfach*	1.26×	[1.129, 1.400]	0.0006	0.0008	-1.47	
rs:schubfach-vs-bd	e2e	canonicalize	medium	schubfach-rs	json-canon-rs	schubfach-rs*	1.66×	[1.535, 1.803]	0.0002	0.0003	-4.23	
schubfach:go-vs-rs	e2e	canonicalize	medium	schubfach	schubfach-rs	schubfach-rs*	2.42×	[2.192, 2.670]	0.0002	0.0003	4.86	
bd:go-vs-rs	e2e	canonicalize	medium	json-canon	json-canon-rs	json-canon-rs*	1.84×	[1.676, 2.011]	0.0002	0.0003	3.97	
go:schubfach-vs-bd	e2e	canonicalize	mixed-prod	schubfach	json-canon	schubfach	1.06×	[0.919, 1.235]	0.4621	0.4932	-0.27	
rs:schubfach-vs-bd	e2e	canonicalize	mixed-prod	schubfach-rs	json-canon-rs	schubfach-rs*	1.34×	[1.237, 1.427]	0.0002	0.0003	-2.87	
schubfach:go-vs-rs	e2e	canonicalize	mixed-prod	schubfach	schubfach-rs	schubfach-rs*	2.55×	[2.301, 2.820]	0.0002	0.0003	4.67	
bd:go-vs-rs	e2e	canonicalize	mixed-prod	json-canon	json-canon-rs	json-canon-rs*	2.02×	[1.787, 2.289]	0.0002	0.0003	2.91	
go:schubfach-vs-bd	e2e	canonicalize	nested-mixed	schubfach	json-canon	schubfach*	1.18×	[1.044, 1.341]	0.0192	0.0234	-0.91	
rs:schubfach-vs-bd	e2e	canonicalize	nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	1.25×	[1.151, 1.340]	0.0002	0.0003	-2.09	
schubfach:go-vs-rs	e2e	canonicalize	nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.47×	[2.206, 2.738]	0.0002	0.0003	4.38	
bd:go-vs-rs	e2e	canonicalize	nested-mixed	json-canon	json-canon-rs	json-canon-rs*	2.33×	[2.114, 2.554]	0.0002	0.0003	4.47	
go:schubfach-vs-bd	e2e	canonicalize	number-heavy	schubfach	json-canon	schubfach*	1.14×	[1.025, 1.307]	0.0404	0.0476	-0.80	
rs:schubfach-vs-bd	e2e	canonicalize	number-heavy	schubfach-rs	json-canon-rs	schubfach-rs*	1.46×	[1.372, 1.572]	0.0002	0.0003	-4.16	
schubfach:go-vs-rs	e2e	canonicalize	number-heavy	schubfach	schubfach-rs	schubfach-rs*	2.74×	[2.427, 3.069]	0.0002	0.0003	4.13	
bd:go-vs-rs	e2e	canonicalize	number-heavy	json-canon	json-canon-rs	json-canon-rs*	2.15×	[2.006, 2.291]	0.0002	0.0003	6.06	
go:schubfach-vs-bd	e2e	canonicalize	numeric-boundary	schubfach	json-canon	schubfach	1.15×	[1.001, 1.301]	0.0552	0.0648	-0.73	
rs:schubfach-vs-bd	e2e	canonicalize	numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	1.28×	[1.164, 1.405]	0.0002	0.0003	-1.88	
schubfach:go-vs-rs	e2e	canonicalize	numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	2.73×	[2.457, 3.089]	0.0002	0.0003	4.59	
bd:go-vs-rs	e2e	canonicalize	numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.45×	[2.178, 2.727]	0.0002	0.0003	4.28	
go:schubfach-vs-bd	e2e	canonicalize	rfc-key-sorting	schubfach	json-canon	schubfach	1.01×	[0.887, 1.160]	0.8894	0.9066	-0.05	
rs:schubfach-vs-bd	e2e	canonicalize	rfc-key-sorting	schubfach-rs	json-canon-rs	schubfach-rs*	1.13×	[1.041, 1.208]	0.0024	0.0031	-1.16	
schubfach:go-vs-rs	e2e	canonicalize	rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.69×	[2.392, 2.972]	0.0002	0.0003	4.34	
bd:go-vs-rs	e2e	canonicalize	rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.41×	[2.161, 2.688]	0.0002	0.0003	4.36	
go:schubfach-vs-bd	e2e	canonicalize	small	schubfach	json-canon	schubfach*	1.22×	[1.092, 1.357]	0.0014	0.0018	-1.33	
rs:schubfach-vs-bd	e2e	canonicalize	small	schubfach-rs	json-canon-rs	schubfach-rs*	1.22×	[1.121, 1.288]	0.0002	0.0003	-1.91	
schubfach:go-vs-rs	e2e	canonicalize	small	schubfach	schubfach-rs	schubfach-rs*	2.54×	[2.305, 2.818]	0.0002	0.0003	4.36	

C

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
bd:go-vs-rs	e2e	canonicalize	small	json-canon	json-canon-rs	json-canon-rs*	2.53×	[2.343, 2.767]	0.0002	0.0003	7.32	
go:schubfach-vs-bd	e2e	canonicalize	surrogate-pair	schubfach	json-canon	schubfach	1.05×	[0.910, 1.221]	0.5325	0.5664	-0.23	
rs:schubfach-vs-bd	e2e	canonicalize	surrogate-pair	schubfach-rs	json-canon-rs	schubfach-rs*	1.09×	[1.016, 1.164]	0.0270	0.0324	-0.83	
schubfach:go-vs-rs	e2e	canonicalize	surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.74×	[2.380, 3.125]	0.0002	0.0003	3.44	
bd:go-vs-rs	e2e	canonicalize	surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.65×	[2.428, 2.901]	0.0002	0.0003	5.75	
go:schubfach-vs-bd	e2e	canonicalize	unicode	schubfach	json-canon	schubfach	1.11×	[0.928, 1.267]	0.1988	0.2219	-0.49	
rs:schubfach-vs-bd	e2e	canonicalize	unicode	schubfach-rs	json-canon-rs	schubfach-rs	1.07×	[0.996, 1.153]	0.0742	0.0861	-0.68	
schubfach:go-vs-rs	e2e	canonicalize	unicode	schubfach	schubfach-rs	schubfach-rs*	2.70×	[2.378, 3.159]	0.0002	0.0003	3.21	
bd:go-vs-rs	e2e	canonicalize	unicode	json-canon	json-canon-rs	json-canon-rs*	2.79×	[2.536, 3.041]	0.0002	0.0003	5.91	
go:schubfach-vs-bd	e2e	canonicalize	verify-whitespace	schubfach	json-canon	schubfach	1.10×	[0.968, 1.249]	0.1806	0.2044	-0.51	
rs:schubfach-vs-bd	e2e	canonicalize	verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	1.34×	[1.259, 1.481]	0.0002	0.0003	-2.58	
schubfach:go-vs-rs	e2e	canonicalize	verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.65×	[2.367, 2.929]	0.0002	0.0003	4.54	
bd:go-vs-rs	e2e	canonicalize	verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	2.18×	[1.932, 2.375]	0.0002	0.0003	4.18	
go:schubfach-vs-bd	e2e	verify	array-2048	schubfach	json-canon	schubfach*	1.31×	[1.171, 1.412]	0.0002	0.0003	-2.24	
rs:schubfach-vs-bd	e2e	verify	array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	1.64×	[1.501, 1.805]	0.0002	0.0003	-3.87	
schubfach:go-vs-rs	e2e	verify	array-2048	schubfach	schubfach-rs	schubfach-rs*	2.19×	[1.994, 2.473]	0.0002	0.0003	4.57	
bd:go-vs-rs	e2e	verify	array-2048	json-canon	json-canon-rs	json-canon-rs*	1.75×	[1.628, 1.872]	0.0002	0.0003	5.42	
go:schubfach-vs-bd	e2e	verify	array-256	schubfach	json-canon	schubfach	1.14×	[1.007, 1.284]	0.0602	0.0704	-0.71	
rs:schubfach-vs-bd	e2e	verify	array-256	schubfach-rs	json-canon-rs	schubfach-rs*	1.42×	[1.290, 1.550]	0.0002	0.0003	-2.41	
schubfach:go-vs-rs	e2e	verify	array-256	schubfach	schubfach-rs	schubfach-rs*	2.40×	[2.203, 2.636]	0.0002	0.0003	5.17	
bd:go-vs-rs	e2e	verify	array-256	json-canon	json-canon-rs	json-canon-rs*	1.92×	[1.706, 2.187]	0.0002	0.0003	3.21	
go:schubfach-vs-bd	e2e	verify	canonical-minimal	schubfach	json-canon	schubfach	1.04×	[0.899, 1.191]	0.6147	0.6474	-0.19	
rs:schubfach-vs-bd	e2e	verify	canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs*	1.14×	[1.048, 1.221]	0.0032	0.0041	-1.18	
schubfach:go-vs-rs	e2e	verify	canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	2.73×	[2.420, 3.050]	0.0002	0.0003	4.15	
bd:go-vs-rs	e2e	verify	canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	2.49×	[2.231, 2.770]	0.0002	0.0003	4.33	
go:schubfach-vs-bd	e2e	verify	control-escapes	schubfach	json-canon	schubfach	1.06×	[0.934, 1.169]	0.3545	0.3862	-0.36	
rs:schubfach-vs-bd	e2e	verify	control-escapes	schubfach-rs	json-canon-rs	schubfach-rs*	1.12×	[1.048, 1.218]	0.0052	0.0066	-1.07	
schubfach:go-vs-rs	e2e	verify	control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.73×	[2.508, 3.005]	0.0002	0.0003	6.59	
bd:go-vs-rs	e2e	verify	control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.59×	[2.303, 2.813]	0.0002	0.0003	4.66	
go:schubfach-vs-bd	e2e	verify	deep-64	schubfach	json-canon	schubfach*	1.17×	[1.056, 1.307]	0.0082	0.0103	-1.04	
rs:schubfach-vs-bd	e2e	verify	deep-64	schubfach-rs	json-canon-rs	schubfach-rs*	1.09×	[1.014, 1.175]	0.0324	0.0383	-0.82	
schubfach:go-vs-rs	e2e	verify	deep-64	schubfach	schubfach-rs	schubfach-rs*	2.70×	[2.398, 2.972]	0.0002	0.0003	4.89	
bd:go-vs-rs	e2e	verify	deep-64	json-canon	json-canon-rs	json-canon-rs*	2.89×	[2.668, 3.128]	0.0002	0.0003	7.07	
go:schubfach-vs-bd	e2e	verify	deep	schubfach	json-canon	schubfach	1.01×	[0.901, 1.136]	0.8542	0.8792	-0.07	
rs:schubfach-vs-bd	e2e	verify	deep	schubfach-rs	json-canon-rs	json-canon-rs	1.02×	[0.837, 1.566]	0.9918	0.9981	0.05	
schubfach:go-vs-rs	e2e	verify	deep	schubfach	schubfach-rs	schubfach-rs*	2.36×	[1.502, 2.839]	0.0002	0.0003	2.86	
bd:go-vs-rs	e2e	verify	deep	json-canon	json-canon-rs	json-canon-rs*	2.44×	[2.087, 2.713]	0.0002	0.0003	4.37	
go:schubfach-vs-bd	e2e	verify	escaped-key-order	schubfach	json-canon	schubfach	1.14×	[0.999, 1.296]	0.0714	0.0832	-0.68	
rs:schubfach-vs-bd	e2e	verify	escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	1.27×	[1.186, 1.364]	0.0002	0.0003	-2.15	
schubfach:go-vs-rs	e2e	verify	escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.55×	[2.281, 2.812]	0.0002	0.0003	4.26	
bd:go-vs-rs	e2e	verify	escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	2.29×	[2.042, 2.518]	0.0002	0.0003	4.33	
go:schubfach-vs-bd	e2e	verify	large	schubfach	json-canon	schubfach*	1.56×	[1.519, 1.595]	0.0002	0.0003	-13.47	
rs:schubfach-vs-bd	e2e	verify	large	schubfach-rs	json-canon-rs	schubfach-rs*	2.47×	[2.280, 2.627]	0.0002	0.0003	-12.99	
schubfach:go-vs-rs	e2e	verify	large	schubfach	schubfach-rs	schubfach-rs*	2.11×	[1.952, 2.248]	0.0002	0.0003	9.68	
bd:go-vs-rs	e2e	verify	large	json-canon	json-canon-rs	json-canon-rs*	1.33×	[1.303, 1.359]	0.0002	0.0003	9.61	
go:schubfach-vs-bd	e2e	verify	long-string	schubfach	json-canon	json-canon	1.03×	[0.925, 1.154]	0.6475	0.6774	0.17	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
rs:schubfach-vs-bd	e2e	verify	long-string	schubfach-rs	json-canon-rs	schubfach-rs*	1.10×	[1.033, 1.175]	0.0086	0.0107	-1.03	
schubfach:go-vs-rs	e2e	verify	long-string	schubfach	schubfach-rs	schubfach-rs*	3.04×	[2.781, 3.308]	0.0002	0.0003	6.48	
bd:go-vs-rs	e2e	verify	long-string	json-canon	json-canon-rs	json-canon-rs*	2.70×	[2.433, 2.943]	0.0002	0.0003	4.99	
go:schubfach-vs-bd	e2e	verify	medium	schubfach	json-canon	schubfach*	1.16×	[1.030, 1.310]	0.0280	0.0335	-0.83	
rs:schubfach-vs-bd	e2e	verify	medium	schubfach-rs	json-canon-rs	schubfach-rs*	1.88×	[1.721, 2.078]	0.0002	0.0003	-5.31	
schubfach:go-vs-rs	e2e	verify	medium	schubfach	schubfach-rs	schubfach-rs*	2.62×	[2.322, 2.946]	0.0002	0.0003	4.57	
bd:go-vs-rs	e2e	verify	medium	json-canon	json-canon-rs	json-canon-rs*	1.61×	[1.482, 1.782]	0.0002	0.0003	3.00	
go:schubfach-vs-bd	e2e	verify	mixed-prod	schubfach	json-canon	schubfach	1.11×	[0.984, 1.242]	0.0988	0.1130	-0.62	
rs:schubfach-vs-bd	e2e	verify	mixed-prod	schubfach-rs	json-canon-rs	schubfach-rs*	1.29×	[1.162, 1.427]	0.0002	0.0003	-1.67	
schubfach:go-vs-rs	e2e	verify	mixed-prod	schubfach	schubfach-rs	schubfach-rs*	2.74×	[2.452, 3.040]	0.0002	0.0003	4.94	
bd:go-vs-rs	e2e	verify	mixed-prod	json-canon	json-canon-rs	json-canon-rs*	2.35×	[2.109, 2.638]	0.0002	0.0003	4.65	
go:schubfach-vs-bd	e2e	verify	nested-mixed	schubfach	json-canon	schubfach	1.14×	[0.998, 1.315]	0.0756	0.0874	-0.67	
rs:schubfach-vs-bd	e2e	verify	nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	1.32×	[1.270, 1.400]	0.0002	0.0003	-4.56	
schubfach:go-vs-rs	e2e	verify	nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.70×	[2.408, 2.931]	0.0002	0.0003	5.14	
bd:go-vs-rs	e2e	verify	nested-mixed	json-canon	json-canon-rs	json-canon-rs*	2.34×	[2.111, 2.610]	0.0002	0.0003	3.68	
go:schubfach-vs-bd	e2e	verify	number-heavy	schubfach	json-canon	schubfach	1.13×	[0.995, 1.295]	0.0876	0.1006	-0.65	
rs:schubfach-vs-bd	e2e	verify	number-heavy	schubfach-rs	json-canon-rs	schubfach-rs*	1.41×	[1.324, 1.528]	0.0002	0.0003	-3.87	
schubfach:go-vs-rs	e2e	verify	number-heavy	schubfach	schubfach-rs	schubfach-rs*	2.81×	[2.487, 3.152]	0.0002	0.0003	4.35	
bd:go-vs-rs	e2e	verify	number-heavy	json-canon	json-canon-rs	json-canon-rs*	2.26×	[2.046, 2.453]	0.0002	0.0003	4.47	
go:schubfach-vs-bd	e2e	verify	numeric-boundary	schubfach	json-canon	schubfach	1.07×	[0.937, 1.229]	0.3477	0.3803	-0.34	
rs:schubfach-vs-bd	e2e	verify	numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	1.24×	[1.134, 1.317]	0.0002	0.0003	-1.94	
schubfach:go-vs-rs	e2e	verify	numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	2.53×	[2.242, 2.784]	0.0002	0.0003	4.21	
bd:go-vs-rs	e2e	verify	numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.19×	[1.959, 2.448]	0.0002	0.0003	3.81	
go:schubfach-vs-bd	e2e	verify	rfc-key-sorting	schubfach	json-canon	json-canon	1.02×	[0.904, 1.148]	0.7157	0.7463	0.13	
rs:schubfach-vs-bd	e2e	verify	rfc-key-sorting	schubfach-rs	json-canon-rs	schubfach-rs*	1.07×	[1.017, 1.125]	0.0160	0.0196	-0.93	
schubfach:go-vs-rs	e2e	verify	rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.68×	[2.417, 2.907]	0.0002	0.0003	5.46	
bd:go-vs-rs	e2e	verify	rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.45×	[2.233, 2.683]	0.0002	0.0003	4.70	
go:schubfach-vs-bd	e2e	verify	small	schubfach	json-canon	schubfach	1.07×	[0.947, 1.218]	0.2871	0.3183	-0.39	
rs:schubfach-vs-bd	e2e	verify	small	schubfach-rs	json-canon-rs	schubfach-rs*	1.15×	[1.075, 1.233]	0.0014	0.0018	-1.35	
schubfach:go-vs-rs	e2e	verify	small	schubfach	schubfach-rs	schubfach-rs*	2.52×	[2.278, 2.771]	0.0002	0.0003	4.44	
bd:go-vs-rs	e2e	verify	small	json-canon	json-canon-rs	json-canon-rs*	2.35×	[2.101, 2.608]	0.0002	0.0003	4.43	
go:schubfach-vs-bd	e2e	verify	surrogate-pair	schubfach	json-canon	schubfach	1.05×	[0.947, 1.161]	0.3587	0.3894	-0.34	
rs:schubfach-vs-bd	e2e	verify	surrogate-pair	schubfach-rs	json-canon-rs	schubfach-rs*	1.15×	[1.056, 1.287]	0.0068	0.0086	-1.00	
schubfach:go-vs-rs	e2e	verify	surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.66×	[2.402, 2.927]	0.0002	0.0003	5.55	
bd:go-vs-rs	e2e	verify	surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.44×	[2.187, 2.663]	0.0002	0.0003	5.37	
go:schubfach-vs-bd	e2e	verify	unicode	schubfach	json-canon	schubfach	1.14×	[0.988, 1.299]	0.0796	0.0917	-0.67	
rs:schubfach-vs-bd	e2e	verify	unicode	schubfach-rs	json-canon-rs	schubfach-rs*	1.09×	[1.021, 1.178]	0.0270	0.0324	-0.83	
schubfach:go-vs-rs	e2e	verify	unicode	schubfach	schubfach-rs	schubfach-rs*	2.48×	[2.247, 2.762]	0.0002	0.0003	4.10	
bd:go-vs-rs	e2e	verify	unicode	json-canon	json-canon-rs	json-canon-rs*	2.60×	[2.306, 2.876]	0.0002	0.0003	4.60	
go:schubfach-vs-bd	e2e	verify	verify-whitespace	schubfach	json-canon	schubfach*	1.24×	[1.096, 1.395]	0.0022	0.0028	-1.25	
rs:schubfach-vs-bd	e2e	verify	verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	1.19×	[1.129, 1.273]	0.0002	0.0003	-2.07	
schubfach:go-vs-rs	e2e	verify	verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.26×	[2.052, 2.532]	0.0002	0.0003	4.00	
bd:go-vs-rs	e2e	verify	verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	2.36×	[2.148, 2.549]	0.0002	0.0003	5.07	
go:schubfach-vs-bd	api	canonicalize	array-2048	schubfach	json-canon	schubfach*	1.57×	[1.558, 1.587]	0.0002	0.0003	-48.58	
rs:schubfach-vs-bd	api	canonicalize	array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	2.51×	[2.423, 2.548]	0.0002	0.0003	-52.77	
schubfach:go-vs-rs	api	canonicalize	array-2048	schubfach	schubfach-rs	schubfach-rs*	1.90×	[1.829, 1.931]	0.0002	0.0003	28.28	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
bd:go-vs-rs	api	canonicalize	array-2048	json-canon	json-canon-rs	json-canon-rs*	1.19×	[1.184, 1.197]	0.0002	0.0003	27.23	
go:schubfach-vs-bd	api	canonicalize	array-256	schubfach	json-canon	schubfach*	1.56×	[1.551, 1.567]	0.0002	0.0003	-78.75	
rs:schubfach-vs-bd	api	canonicalize	array-256	schubfach-rs	json-canon-rs	schubfach-rs*	2.20×	[2.160, 2.258]	0.0002	0.0003	-32.70	
schubfach:go-vs-rs	api	canonicalize	array-256	schubfach	schubfach-rs	schubfach-rs*	1.76×	[1.732, 1.803]	0.0002	0.0003	30.68	
bd:go-vs-rs	api	canonicalize	array-256	json-canon	json-canon-rs	json-canon-rs*	1.25×	[1.234, 1.262]	0.0002	0.0003	18.20	
go:schubfach-vs-bd	api	canonicalize	canonical-minimal	schubfach	json-canon	schubfach*	1.54×	[1.529, 1.555]	0.0002	0.0003	-47.74	
rs:schubfach-vs-bd	api	canonicalize	canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs*	2.10×	[1.991, 2.205]	0.0002	0.0003	-12.63	
schubfach:go-vs-rs	api	canonicalize	canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	1.63×	[1.547, 1.684]	0.0002	0.0003	12.20	
bd:go-vs-rs	api	canonicalize	canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	1.19×	[1.141, 1.219]	0.0002	0.0003	5.60	
go:schubfach-vs-bd	api	canonicalize	control-escapes	schubfach	json-canon	json-canon*	1.06×	[1.035, 1.071]	0.0004	0.0005	2.81	
rs:schubfach-vs-bd	api	canonicalize	control-escapes	schubfach-rs	json-canon-rs	json-canon-rs	1.02×	[0.961, 1.063]	0.4467	0.4784	0.37	
schubfach:go-vs-rs	api	canonicalize	control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.15×	[2.092, 2.191]	0.0002	0.0003	33.47	
bd:go-vs-rs	api	canonicalize	control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.08×	[1.950, 2.152]	0.0002	0.0003	16.71	
go:schubfach-vs-bd	api	canonicalize	deep-64	schubfach	json-canon	json-canon	1.00×	[0.999, 1.006]	0.3325	0.3660	0.44	
rs:schubfach-vs-bd	api	canonicalize	deep-64	schubfach-rs	json-canon-rs	schubfach-rs	1.00×	[0.979, 1.061]	1.0000	1.0000	-0.01	
schubfach:go-vs-rs	api	canonicalize	deep-64	schubfach	schubfach-rs	schubfach-rs*	1.49×	[1.485, 1.499]	0.0002	0.0003	71.37	
bd:go-vs-rs	api	canonicalize	deep-64	json-canon	json-canon-rs	json-canon-rs*	1.49×	[1.402, 1.521]	0.0002	0.0003	11.37	
go:schubfach-vs-bd	api	canonicalize	deep	schubfach	json-canon	schubfach	1.00×	[0.998, 1.006]	0.3479	0.3803	-0.44	
rs:schubfach-vs-bd	api	canonicalize	deep	schubfach-rs	json-canon-rs	json-canon-rs*	1.05×	[1.014, 1.094]	0.0290	0.0346	1.03	
schubfach:go-vs-rs	api	canonicalize	deep	schubfach	schubfach-rs	schubfach-rs*	1.21×	[1.158, 1.244]	0.0002	0.0003	4.93	
bd:go-vs-rs	api	canonicalize	deep	json-canon	json-canon-rs	json-canon-rs*	1.27×	[1.243, 1.282]	0.0002	0.0003	15.49	
go:schubfach-vs-bd	api	canonicalize	escaped-key-order	schubfach	json-canon	schubfach*	1.60×	[1.593, 1.607]	0.0002	0.0003	-80.96	
rs:schubfach-vs-bd	api	canonicalize	escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	2.49×	[2.396, 2.546]	0.0002	0.0003	-27.97	
schubfach:go-vs-rs	api	canonicalize	escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.16×	[2.077, 2.191]	0.0002	0.0003	41.04	
bd:go-vs-rs	api	canonicalize	escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	1.38×	[1.366, 1.408]	0.0002	0.0003	19.98	
go:schubfach-vs-bd	api	canonicalize	large	schubfach	json-canon	schubfach*	1.72×	[1.707, 1.724]	0.0002	0.0003	-92.98	
rs:schubfach-vs-bd	api	canonicalize	large	schubfach-rs	json-canon-rs	schubfach-rs*	3.39×	[3.272, 3.453]	0.0002	0.0003	-40.35	
schubfach:go-vs-rs	api	canonicalize	large	schubfach	schubfach-rs	schubfach-rs*	2.07×	[1.995, 2.095]	0.0002	0.0003	39.93	
bd:go-vs-rs	api	canonicalize	large	json-canon	json-canon-rs	json-canon-rs*	1.05×	[1.029, 1.058]	0.0002	0.0003	2.84	
go:schubfach-vs-bd	api	canonicalize	long-string	schubfach	json-canon	json-canon	1.01×	[0.997, 1.025]	0.1908	0.2137	0.60	
rs:schubfach-vs-bd	api	canonicalize	long-string	schubfach-rs	json-canon-rs	json-canon-rs	1.00×	[0.949, 1.030]	0.8706	0.8927	0.11	
schubfach:go-vs-rs	api	canonicalize	long-string	schubfach	schubfach-rs	schubfach-rs*	4.41×	[4.325, 4.482]	0.0002	0.0003	56.30	
bd:go-vs-rs	api	canonicalize	long-string	json-canon	json-canon-rs	json-canon-rs*	4.39×	[4.160, 4.494]	0.0002	0.0003	58.45	
go:schubfach-vs-bd	api	canonicalize	medium	schubfach	json-canon	schubfach*	1.69×	[1.678, 1.703]	0.0002	0.0003	-53.37	
rs:schubfach-vs-bd	api	canonicalize	medium	schubfach-rs	json-canon-rs	schubfach-rs*	2.97×	[2.936, 2.996]	0.0002	0.0003	-69.85	
schubfach:go-vs-rs	api	canonicalize	medium	schubfach	schubfach-rs	schubfach-rs*	1.97×	[1.952, 1.987]	0.0002	0.0003	64.66	
bd:go-vs-rs	api	canonicalize	medium	json-canon	json-canon-rs	json-canon-rs*	1.12×	[1.112, 1.134]	0.0002	0.0003	10.37	
go:schubfach-vs-bd	api	canonicalize	mixed-prod	schubfach	json-canon	schubfach*	1.31×	[1.302, 1.317]	0.0002	0.0003	-38.13	
rs:schubfach-vs-bd	api	canonicalize	mixed-prod	schubfach-rs	json-canon-rs	schubfach-rs*	1.83×	[1.791, 1.856]	0.0002	0.0003	-29.73	
schubfach:go-vs-rs	api	canonicalize	mixed-prod	schubfach	schubfach-rs	schubfach-rs*	2.06×	[2.017, 2.082]	0.0002	0.0003	56.22	
bd:go-vs-rs	api	canonicalize	mixed-prod	json-canon	json-canon-rs	json-canon-rs*	1.48×	[1.455, 1.489]	0.0002	0.0003	32.80	
go:schubfach-vs-bd	api	canonicalize	nested-mixed	schubfach	json-canon	schubfach*	1.41×	[1.399, 1.413]	0.0002	0.0003	-51.38	
rs:schubfach-vs-bd	api	canonicalize	nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	1.92×	[1.844, 1.959]	0.0002	0.0003	-20.61	
schubfach:go-vs-rs	api	canonicalize	nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.14×	[2.063, 2.178]	0.0002	0.0003	37.76	
bd:go-vs-rs	api	canonicalize	nested-mixed	json-canon	json-canon-rs	json-canon-rs*	1.57×	[1.549, 1.596]	0.0002	0.0003	29.76	
go:schubfach-vs-bd	api	canonicalize	number-heavy	schubfach	json-canon	schubfach*	1.56×	[1.547, 1.573]	0.0004	0.0005	-43.14	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
rs:schubfach-vs-bd	api	canonicalize	number-heavy	schubfach-rs	json-canon-rs	schubfach-rs*	8.27×	[8.179, 8.374]	0.0002	0.0003	-110.14	
schubfach:go-vs-rs	api	canonicalize	number-heavy	schubfach	schubfach-rs	schubfach-rs*	9.72×	[9.613, 9.834]	0.0002	0.0003	117.62	
bd:go-vs-rs	api	canonicalize	number-heavy	json-canon	json-canon-rs	json-canon-rs*	1.83×	[1.817, 1.849]	0.0002	0.0003	56.61	
go:schubfach-vs-bd	api	canonicalize	numeric-boundary	schubfach	json-canon	schubfach*	1.49×	[1.482, 1.502]	0.0002	0.0003	-48.06	
rs:schubfach-vs-bd	api	canonicalize	numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	9.32×	[9.208, 9.424]	0.0002	0.0003	-164.23	
schubfach:go-vs-rs	api	canonicalize	numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	13.17×	[13.007, 13.323]	0.0002	0.0003	148.54	
bd:go-vs-rs	api	canonicalize	numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.11×	[2.097, 2.123]	0.0002	0.0003	87.16	
go:schubfach-vs-bd	api	canonicalize	rfc-key-sorting	schubfach	json-canon	json-canon*	1.02×	[1.011, 1.023]	0.0002	0.0003	2.41	
rs:schubfach-vs-bd	api	canonicalize	rfc-key-sorting	schubfach-rs	json-canon-rs	json-canon-rs	1.01×	[0.967, 1.044]	0.7365	0.7629	0.15	
schubfach:go-vs-rs	api	canonicalize	rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.28×	[2.235, 2.341]	0.0002	0.0003	43.48	
bd:go-vs-rs	api	canonicalize	rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.26×	[2.189, 2.326]	0.0002	0.0003	33.49	
go:schubfach-vs-bd	api	canonicalize	small	schubfach	json-canon	schubfach*	1.34×	[1.324, 1.345]	0.0002	0.0003	-35.59	
rs:schubfach-vs-bd	api	canonicalize	small	schubfach-rs	json-canon-rs	schubfach-rs*	1.75×	[1.712, 1.784]	0.0002	0.0003	-22.01	
schubfach:go-vs-rs	api	canonicalize	small	schubfach	schubfach-rs	schubfach-rs*	1.97×	[1.941, 2.005]	0.0002	0.0003	40.48	
bd:go-vs-rs	api	canonicalize	small	json-canon	json-canon-rs	json-canon-rs*	1.51×	[1.487, 1.530]	0.0002	0.0003	29.11	
go:schubfach-vs-bd	api	canonicalize	surrogate-pair	schubfach	json-canon	json-canon*	1.01×	[1.008, 1.019]	0.0002	0.0003	2.09	
rs:schubfach-vs-bd	api	canonicalize	surrogate-pair	schubfach-rs	json-canon-rs	json-canon-rs	1.00×	[0.943, 1.075]	0.9078	0.9224	0.06	
schubfach:go-vs-rs	api	canonicalize	surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.08×	[1.932, 2.155]	0.0002	0.0003	18.14	
bd:go-vs-rs	api	canonicalize	surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.06×	[1.949, 2.138]	0.0002	0.0003	19.65	
go:schubfach-vs-bd	api	canonicalize	unicode	schubfach	json-canon	json-canon	1.00×	[0.999, 1.009]	0.1816	0.2048	0.63	
rs:schubfach-vs-bd	api	canonicalize	unicode	schubfach-rs	json-canon-rs	schubfach-rs*	1.03×	[1.005, 1.045]	0.0228	0.0277	-1.13	
schubfach:go-vs-rs	api	canonicalize	unicode	schubfach	schubfach-rs	schubfach-rs*	1.05×	[1.030, 1.066]	0.0002	0.0003	2.55	
bd:go-vs-rs	api	canonicalize	unicode	json-canon	json-canon-rs	json-canon-rs*	1.02×	[1.007, 1.030]	0.0060	0.0076	1.45	
go:schubfach-vs-bd	api	canonicalize	verify-whitespace	schubfach	json-canon	schubfach*	1.71×	[1.699, 1.723]	0.0002	0.0003	-56.22	
rs:schubfach-vs-bd	api	canonicalize	verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	2.90×	[2.831, 2.980]	0.0002	0.0003	-30.28	
schubfach:go-vs-rs	api	canonicalize	verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.28×	[2.235, 2.325]	0.0002	0.0003	47.39	
bd:go-vs-rs	api	canonicalize	verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	1.34×	[1.317, 1.365]	0.0002	0.0003	15.69	
go:schubfach-vs-bd	api	verify	canonical/array-2048	schubfach	json-canon	schubfach*	1.62×	[1.611, 1.628]	0.0002	0.0003	-76.18	
rs:schubfach-vs-bd	api	verify	canonical/array-2048	schubfach-rs	json-canon-rs	schubfach-rs*	2.54×	[2.508, 2.556]	0.0002	0.0003	-81.07	
schubfach:go-vs-rs	api	verify	canonical/array-2048	schubfach	schubfach-rs	schubfach-rs*	1.84×	[1.826, 1.857]	0.0002	0.0003	71.85	
bd:go-vs-rs	api	verify	canonical/array-2048	json-canon	json-canon-rs	json-canon-rs*	1.18×	[1.171, 1.186]	0.0002	0.0003	21.36	
go:schubfach-vs-bd	api	verify	canonical/array-256	schubfach	json-canon	schubfach*	1.52×	[1.512, 1.532]	0.0002	0.0003	-47.19	
rs:schubfach-vs-bd	api	verify	canonical/array-256	schubfach-rs	json-canon-rs	schubfach-rs*	2.24×	[2.228, 2.260]	0.0002	0.0003	-93.79	
schubfach:go-vs-rs	api	verify	canonical/array-256	schubfach	schubfach-rs	schubfach-rs*	1.88×	[1.864, 1.891]	0.0002	0.0003	75.82	
bd:go-vs-rs	api	verify	canonical/array-256	json-canon	json-canon-rs	json-canon-rs*	1.27×	[1.265, 1.283]	0.0002	0.0003	28.28	
go:schubfach-vs-bd	api	verify	canonical/canonical-minimal	schubfach	json-canon	schubfach*	1.52×	[1.511, 1.523]	0.0002	0.0003	-86.58	
rs:schubfach-vs-bd	api	verify	canonical/canonical-minimal	schubfach-rs	json-canon-rs	schubfach-rs*	2.13×	[2.083, 2.198]	0.0002	0.0003	-19.04	
schubfach:go-vs-rs	api	verify	canonical/canonical-minimal	schubfach	schubfach-rs	schubfach-rs*	1.75×	[1.709, 1.763]	0.0002	0.0003	42.49	
bd:go-vs-rs	api	verify	canonical/canonical-minimal	json-canon	json-canon-rs	json-canon-rs*	1.24×	[1.206, 1.265]	0.0002	0.0003	8.96	
go:schubfach-vs-bd	api	verify	canonical/control-escapes	schubfach	json-canon	json-canon*	1.04×	[1.036, 1.048]	0.0002	0.0003	5.85	
rs:schubfach-vs-bd	api	verify	canonical/control-escapes	schubfach-rs	json-canon-rs	json-canon-rs	1.01×	[0.965, 1.053]	0.5379	0.5703	0.29	
schubfach:go-vs-rs	api	verify	canonical/control-escapes	schubfach	schubfach-rs	schubfach-rs*	2.09×	[2.033, 2.142]	0.0002	0.0003	33.29	
bd:go-vs-rs	api	verify	canonical/control-escapes	json-canon	json-canon-rs	json-canon-rs*	2.04×	[1.947, 2.092]	0.0002	0.0003	24.89	
go:schubfach-vs-bd	api	verify	canonical/deep-64	schubfach	json-canon	schubfach	1.00×	[0.999, 1.010]	0.1578	0.1793	-0.66	
rs:schubfach-vs-bd	api	verify	canonical/deep-64	schubfach-rs	json-canon-rs	json-canon-rs*	1.02×	[1.017, 1.029]	0.0004	0.0005	3.09	
schubfach:go-vs-rs	api	verify	canonical/deep-64	schubfach	schubfach-rs	schubfach-rs*	1.52×	[1.515, 1.531]	0.0004	0.0005	60.12	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	N
bd:go-vs-rs	api	verify	canonical/deep-64	json-canon	json-canon-rs	json-canon-rs*	1.56×	[1.555, 1.575]	0.0002	0.0003	58.27	
go:schubfach-vs-bd	api	verify	canonical/deep	schubfach	json-canon	schubfach	1.01×	[1.000, 1.013]	0.1006	0.1147	-0.78	
rs:schubfach-vs-bd	api	verify	canonical/deep	schubfach-rs	json-canon-rs	json-canon-rs*	1.02×	[1.013, 1.034]	0.0014	0.0018	1.87	
schubfach:go-vs-rs	api	verify	canonical/deep	schubfach	schubfach-rs	schubfach-rs*	1.31×	[1.298, 1.323]	0.0002	0.0003	25.23	
bd:go-vs-rs	api	verify	canonical/deep	json-canon	json-canon-rs	json-canon-rs*	1.35×	[1.340, 1.361]	0.0002	0.0003	32.48	
go:schubfach-vs-bd	api	verify	canonical/escaped-key-order	schubfach	json-canon	schubfach*	1.63×	[1.629, 1.640]	0.0002	0.0003	-109.67	
rs:schubfach-vs-bd	api	verify	canonical/escaped-key-order	schubfach-rs	json-canon-rs	schubfach-rs*	2.48×	[2.380, 2.563]	0.0002	0.0003	-26.64	
schubfach:go-vs-rs	api	verify	canonical/escaped-key-order	schubfach	schubfach-rs	schubfach-rs*	2.08×	[1.992, 2.132]	0.0002	0.0003	26.56	
bd:go-vs-rs	api	verify	canonical/escaped-key-order	json-canon	json-canon-rs	json-canon-rs*	1.37×	[1.350, 1.389]	0.0002	0.0003	22.64	
go:schubfach-vs-bd	api	verify	canonical/large	schubfach	json-canon	schubfach*	1.74×	[1.733, 1.750]	0.0002	0.0003	-80.15	
rs:schubfach-vs-bd	api	verify	canonical/large	schubfach-rs	json-canon-rs	schubfach-rs*	3.27×	[3.234, 3.317]	0.0002	0.0003	-65.45	
schubfach:go-vs-rs	api	verify	canonical/large	schubfach	schubfach-rs	schubfach-rs*	1.95×	[1.933, 1.973]	0.0002	0.0003	72.25	
bd:go-vs-rs	api	verify	canonical/large	json-canon	json-canon-rs	json-canon-rs*	1.04×	[1.029, 1.050]	0.0002	0.0003	3.31	
go:schubfach-vs-bd	api	verify	canonical/long-string	schubfach	json-canon	json-canon	1.00×	[0.991, 1.019]	0.8480	0.8757	0.10	
rs:schubfach-vs-bd	api	verify	canonical/long-string	schubfach-rs	json-canon-rs	json-canon-rs*	1.01×	[1.001, 1.021]	0.0238	0.0288	0.78	
schubfach:go-vs-rs	api	verify	canonical/long-string	schubfach	schubfach-rs	schubfach-rs*	4.54×	[4.502, 4.623]	0.0002	0.0003	54.59	
bd:go-vs-rs	api	verify	canonical/long-string	json-canon	json-canon-rs	json-canon-rs*	4.57×	[4.527, 4.636]	0.0002	0.0003	87.49	
go:schubfach-vs-bd	api	verify	canonical/medium	schubfach	json-canon	schubfach*	1.68×	[1.675, 1.694]	0.0002	0.0003	-82.24	
rs:schubfach-vs-bd	api	verify	canonical/medium	schubfach-rs	json-canon-rs	schubfach-rs*	2.64×	[2.000, 2.890]	0.0002	0.0003	-8.93	
schubfach:go-vs-rs	api	verify	canonical/medium	schubfach	schubfach-rs	schubfach-rs*	1.73×	[1.320, 1.901]	0.0002	0.0003	4.02	
bd:go-vs-rs	api	verify	canonical/medium	json-canon	json-canon-rs	json-canon-rs*	1.11×	[1.098, 1.114]	0.0002	0.0003	12.73	
go:schubfach-vs-bd	api	verify	canonical/mixed-prod	schubfach	json-canon	schubfach*	1.30×	[1.293, 1.320]	0.0002	0.0003	-21.08	
rs:schubfach-vs-bd	api	verify	canonical/mixed-prod	schubfach-rs	json-canon-rs	schubfach-rs*	1.87×	[1.841, 1.885]	0.0002	0.0003	-51.10	
schubfach:go-vs-rs	api	verify	canonical/mixed-prod	schubfach	schubfach-rs	schubfach-rs*	2.08×	[2.055, 2.108]	0.0002	0.0003	51.04	
bd:go-vs-rs	api	verify	canonical/mixed-prod	json-canon	json-canon-rs	json-canon-rs*	1.46×	[1.446, 1.474]	0.0002	0.0003	31.27	
go:schubfach-vs-bd	api	verify	canonical/nested-mixed	schubfach	json-canon	schubfach*	1.41×	[1.397, 1.415]	0.0002	0.0003	-42.58	
rs:schubfach-vs-bd	api	verify	canonical/nested-mixed	schubfach-rs	json-canon-rs	schubfach-rs*	1.97×	[1.929, 2.023]	0.0002	0.0003	-26.26	
schubfach:go-vs-rs	api	verify	canonical/nested-mixed	schubfach	schubfach-rs	schubfach-rs*	2.22×	[2.178, 2.272]	0.0002	0.0003	44.55	
bd:go-vs-rs	api	verify	canonical/nested-mixed	json-canon	json-canon-rs	json-canon-rs*	1.58×	[1.566, 1.605]	0.0002	0.0003	35.21	
go:schubfach-vs-bd	api	verify	canonical/number-heavy	schubfach	json-canon	schubfach*	1.54×	[1.526, 1.560]	0.0002	0.0003	-33.41	
rs:schubfach-vs-bd	api	verify	canonical/number-heavy	schubfach-rs	json-canon-rs	schubfach-rs*	8.40×	[8.278, 8.707]	0.0002	0.0003	-37.51	
schubfach:go-vs-rs	api	verify	canonical/number-heavy	schubfach	schubfach-rs	schubfach-rs*	10.12×	[9.998, 10.254]	0.0002	0.0003	87.68	
bd:go-vs-rs	api	verify	canonical/number-heavy	json-canon	json-canon-rs	json-canon-rs*	1.86×	[1.795, 1.883]	0.0002	0.0003	30.69	
go:schubfach-vs-bd	api	verify	canonical/numeric-boundary	schubfach	json-canon	schubfach*	1.47×	[1.457, 1.482]	0.0002	0.0003	-34.17	
rs:schubfach-vs-bd	api	verify	canonical/numeric-boundary	schubfach-rs	json-canon-rs	schubfach-rs*	9.40×	[9.234, 9.774]	0.0004	0.0005	-125.14	
schubfach:go-vs-rs	api	verify	canonical/numeric-boundary	schubfach	schubfach-rs	schubfach-rs*	13.51×	[13.275, 14.041]	0.0002	0.0003	145.88	
bd:go-vs-rs	api	verify	canonical/numeric-boundary	json-canon	json-canon-rs	json-canon-rs*	2.11×	[2.092, 2.129]	0.0002	0.0003	58.87	
go:schubfach-vs-bd	api	verify	canonical/rfc-key-sorting	schubfach	json-canon	json-canon*	1.01×	[1.008, 1.019]	0.0004	0.0005	2.10	
rs:schubfach-vs-bd	api	verify	canonical/rfc-key-sorting	schubfach-rs	json-canon-rs	json-canon-rs	1.02×	[0.980, 1.064]	0.3703	0.4007	0.42	
schubfach:go-vs-rs	api	verify	canonical/rfc-key-sorting	schubfach	schubfach-rs	schubfach-rs*	2.13×	[2.042, 2.167]	0.0002	0.0003	36.15	
bd:go-vs-rs	api	verify	canonical/rfc-key-sorting	json-canon	json-canon-rs	json-canon-rs*	2.15×	[2.063, 2.211]	0.0002	0.0003	26.94	
go:schubfach-vs-bd	api	verify	canonical/small	schubfach	json-canon	schubfach*	1.35×	[1.343, 1.355]	0.0002	0.0003	-59.48	
rs:schubfach-vs-bd	api	verify	canonical/small	schubfach-rs	json-canon-rs	schubfach-rs*	1.71×	[1.642, 1.775]	0.0002	0.0003	-11.95	
schubfach:go-vs-rs	api	verify	canonical/small	schubfach	schubfach-rs	schubfach-rs*	1.85×	[1.781, 1.902]	0.0002	0.0003	21.69	
bd:go-vs-rs	api	verify	canonical/small	json-canon	json-canon-rs	json-canon-rs*	1.46×	[1.423, 1.489]	0.0002	0.0003	16.94	
go:schubfach-vs-bd	api	verify	canonical/surrogate-pair	schubfach	json-canon	json-canon*	1.01×	[1.003, 1.011]	0.0060	0.0076	1.39	

Co

Pair	Track	Mode	Workload	Impl A	Impl B	Winner	Speedup	CI_{95}	p	p_{adj}	d	No
rs:schubfach-vs-bd	api	verify	canonical/surrogate-pair	schubfach-rs	json-canon-rs	json-canon-rs	1.00×	[0.950, 1.065]	0.9600	0.9723	0.02	
schubfach:go-vs-rs	api	verify	canonical/surrogate-pair	schubfach	schubfach-rs	schubfach-rs*	2.12×	[1.991, 2.193]	0.0002	0.0003	20.09	
bd:go-vs-rs	api	verify	canonical/surrogate-pair	json-canon	json-canon-rs	json-canon-rs*	2.10×	[2.011, 2.165]	0.0002	0.0003	25.23	
go:schubfach-vs-bd	api	verify	canonical/unicode	schubfach	json-canon	json-canon	1.00×	[0.998, 1.007]	0.3163	0.3494	0.45	
rs:schubfach-vs-bd	api	verify	canonical/unicode	schubfach-rs	json-canon-rs	schubfach-rs	1.12×	[0.975, 1.643]	0.9872	0.9966	-0.38	
schubfach:go-vs-rs	api	verify	canonical/unicode	schubfach	schubfach-rs	schubfach-rs*	1.02×	[1.000, 1.034]	0.0296	0.0351	1.04	
bd:go-vs-rs	api	verify	canonical/unicode	json-canon	json-canon-rs	json-canon	1.10×	[0.961, 1.645]	1.0000	1.0000	-0.32	
go:schubfach-vs-bd	api	verify	canonical/verify-whitespace	schubfach	json-canon	schubfach*	1.70×	[1.688, 1.707]	0.0004	0.0005	-76.03	
rs:schubfach-vs-bd	api	verify	canonical/verify-whitespace	schubfach-rs	json-canon-rs	schubfach-rs*	2.92×	[2.831, 3.020]	0.0002	0.0003	-22.49	
schubfach:go-vs-rs	api	verify	canonical/verify-whitespace	schubfach	schubfach-rs	schubfach-rs*	2.36×	[2.299, 2.412]	0.0002	0.0003	44.61	
bd:go-vs-rs	api	verify	canonical/verify-whitespace	json-canon	json-canon-rs	json-canon-rs*	1.37×	[1.333, 1.399]	0.0002	0.0003	13.15	
go:schubfach-vs-bd	api	verify	noncanonical/control-escapes	schubfach	json-canon	json-canon*	1.05×	[1.042, 1.053]	0.0002	0.0003	7.46	
go:schubfach-vs-bd	api	verify	noncanonical/escaped-key-order	schubfach	json-canon	schubfach*	1.60×	[1.594, 1.610]	0.0002	0.0003	-75.78	
go:schubfach-vs-bd	api	verify	noncanonical/large	schubfach	json-canon	schubfach*	1.72×	[1.716, 1.735]	0.0002	0.0003	-68.62	
go:schubfach-vs-bd	api	verify	noncanonical/medium	schubfach	json-canon	schubfach*	1.68×	[1.670, 1.688]	0.0002	0.0003	-78.68	
go:schubfach-vs-bd	api	verify	noncanonical/mixed-prod	schubfach	json-canon	schubfach*	1.31×	[1.298, 1.315]	0.0002	0.0003	-33.85	
go:schubfach-vs-bd	api	verify	noncanonical/nested-mixed	schubfach	json-canon	schubfach*	1.41×	[1.407, 1.415]	0.0002	0.0003	-94.73	
go:schubfach-vs-bd	api	verify	noncanonical/number-heavy	schubfach	json-canon	schubfach*	1.55×	[1.536, 1.562]	0.0004	0.0005	-44.32	
go:schubfach-vs-bd	api	verify	noncanonical/numeric-boundary	schubfach	json-canon	schubfach*	1.49×	[1.483, 1.499]	0.0002	0.0003	-62.37	
go:schubfach-vs-bd	api	verify	noncanonical/rfc-key-sorting	schubfach	json-canon	json-canon	1.00×	[0.997, 1.008]	0.4055	0.4372	0.38	
go:schubfach-vs-bd	api	verify	noncanonical/small	schubfach	json-canon	schubfach*	1.35×	[1.339, 1.353]	0.0002	0.0003	-46.50	
go:schubfach-vs-bd	api	verify	noncanonical/surrogate-pair	schubfach	json-canon	json-canon*	1.01×	[1.002, 1.012]	0.0154	0.0189	1.23	
go:schubfach-vs-bd	api	verify	noncanonical/unicode	schubfach	json-canon	schubfach	1.00×	[0.997, 1.007]	0.4217	0.4532	-0.37	
go:schubfach-vs-bd	api	verify	noncanonical/verify-whitespace	schubfach	json-canon	schubfach*	1.72×	[1.717, 1.732]	0.0002	0.0003	-97.39	

*Significant after Benjamini-Hochberg FDR correction at $q = 0.05$.
d = Cohen's d: $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster. Track: e2e = end-to-end CLI.
Noise = CV × mean of faster impl. MDE = minimum detectable effect at 80% power (z-approx).
Rust allocs/op and B/op measured via counting global allocator (§8).

B Reproducibility Checklist

This appendix documents the information required to reproduce the benchmark results reported in this paper.

Software Versions

- Go 1.25.6 (`go version go1.25.6 linux/amd64`)
- Rust 1.94.0 (`rustc 1.94.0`)
- Node.js v23.3.0 (V8 12.9.202.28-node.11) — oracle generation
- Linux 6.8.0-101-generic (Ubuntu)

Hardware Specification

- CPU: Intel Core i7-12700K (8 P-cores at 3.6–5.0 GHz, 4 E-cores at 2.7–3.8 GHz)
- RAM: 64 GB DDR4
- Architecture: x86-64
- CPU governor: `performance`

Oracle Vector Checksums (SHA-256)

- `golden_vectors.csv` (54,445 vectors):
  `593bdecbe0dccbc182bc3baf570b716887db25739fc61b7808764ecb966d5636`
- `golden_stress_vectors.csv` (231,917 vectors):
  `287d21ac87e5665550f1baf86038302a0afc67a74a020dffb872f1a93b26d410`

Reproduction Commands

```
git clone <repository-url> && cd jcs-bench-lab
go run ./cmd/lab setup
go run ./cmd/lab conformance
go run ./cmd/lab bench-api -count 10
go run ./cmd/lab bench-cli -repeats 15
go run ./cmd/lab fuzz -cases 2000
go run ./cmd/lab stats -resamples 5000
go run ./cmd/lab gate
go run scripts/generate-paper-tables.go
```

Build Configuration

- Go implementations: default `go build` flags
- Rust implementations: `opt-level=3`, `lto=true`, `codegen-units=1`, `strip=true`
- Rust API benchmarks use a counting global allocator (`std::alloc::GlobalAlloc`) to measure per-operation allocation bytes and counts

Data Availability

All raw benchmark data, statistical reports, oracle vectors, and the benchmark infrastructure are archived at Zenodo.[3]

[3]Zenodo DOI to be inserted for camera-ready version.